

CeliLac: Homepage Profissional

Tema, navegação, dados, estados, acessibilidade e testes, passo a passo

Como usar este material

Este material continua o **CeliLac: Splash e Onboarding Profissionais**. Lá, o usuário chegava à **HomePage**, que era só um texto no centro da tela. Aqui, transformamos essa tela no **centro do aplicativo**:

```
Splash → Onboarding → AppShell (barra de navegação inferior)
                        |
                        |— Início    ← HomePage (este material)
                        |— Explorar  ← "em breve"
                        |— Favoritos ← "em breve"
                        |— Perfil    ← "em breve"
```

Cada parte alterna a ideia por trás do código (que vale para qualquer projeto) com o código implementado no CeliLac. Ao longo do texto, alguns trechos são marcados por um rótulo em negrito:

- **Atenção:** armadilhas comuns.
- **No seu projeto:** o que adaptar para o seu app.
- **Execute:** o momento de rodar o app e verificar o resultado.

Como conteúdo complementar, para quem quiser se aprofundar, o material apresenta testes automatizados (Parte 7).

O objetivo continua o mesmo: **entender cada decisão** para aplicá-la no seu projeto, com a sua marca, o seu domínio e os seus dados. O CeliLac mostra estabelecimentos. O seu app pode mostrar livros, treinos, eventos ou pedidos: a estrutura é a mesma.

Todo o código deste material foi compilado com **flutter analyze** (sem avisos) e validado com 10 testes automatizados (Parte 7), no Flutter 3.47 / Dart 3.13.

Resultado esperado

Topo da Home Lista "Perto de você"

 Topo da Home  Lista de estabelecimentos

Parte 0: Antes do código

0.1 Conceito: o papel da Home

A Home é a tela que o usuário vê **todos os dias**. A Splash e o onboarding aparecem uma vez; a Home aparece em toda abertura do app. Por isso, ela precisa responder rapidamente a três perguntas:

1. **Onde estou?** Identidade (logo, nome, cores) e contexto (saudação).
2. **O que posso fazer aqui?** As ações principais, visíveis sem procurar.
3. **Qual o próximo passo?** Uma sugestão clara do que fazer agora.

Uma Home que tenta mostrar **tudo** não responde a nenhuma delas. O trabalho de design é **escolher** o que entra e, principalmente, o que fica de fora.

0.2 A Home cumpre as promessas do onboarding

O onboarding fez três promessas ao usuário. Uma Home profissional **entrega** essas promessas logo na primeira tela, senão o onboarding vira propaganda enganosa.

Página do onboarding	Promessa	Onde a Home entrega
1. Descoberta	"Encontre opções adequadas a você"	Busca, categorias e lista "Perto de você"
2. Informação	"Entenda antes de escolher"	Cartões com selos "Sem glúten" / "Sem lactose", distância e nota
3. Personalização	"Uma experiência mais relevante"	Cartão "Complete seu perfil alimentar"

No seu projeto: antes de desenhar a Home, releia as frases do seu onboarding e faça esta mesma tabela. Cada promessa precisa de um lugar na Home.

0.3 Estrutura e hierarquia

[logo] CeliLac

Bom dia!

—

O que você procura hoje?

Buscar produtos ou lugares

Complete seu perfil
[Completar perfil]

Categorias
(Rest.) (Pad.) (Café) (Merc.)

Perto de você

Ver todos

Café Aconchego ★ 4,8

Café · Centro · 350 m

[✓ Sem glúten] [✓ Sem lact.]

...

Início

Explorar

Favoritos

Perfil

← SliverAppBar (fixa no topo)

← 1. Contexto: saudação
(assinatura dourada)

← 2. Ação principal: busca

← 3. Próximo passo (destaque)

← 4. Atalhos

← 5. Conteúdo (dados assíncronos)

← NavigationBar (fixa)

Por que essa ordem? O olho percorre a tela de cima para baixo. O que está no topo é visto por **todos**; o que está no fim, só por quem rola. Por isso:

- **Busca no topo:** é a ação mais frequente de quem procura algo ("Descoberta").
- **Cartão de perfil logo depois:** é o passo que **melhora todo o resto** do app. Como é uma ação única (feita uma vez), ele aparece com destaque, mas não acima da busca.
- **Categorias antes da lista:** atalhos curtos e previsíveis. O usuário que não sabe o nome do lugar começa por elas.
- **Lista por último:** é o conteúdo que **pode crescer**. Colocá-la no fim permite rolar sem empurrar as ações para fora da tela.

0.4 O que fica de fora (e por quê)

Tão importante quanto o que entra é o que **não** entra:

Tentação	Por que não agora
Ícone de notificações no topo	O app não tem notificações. Botão que não faz nada destrói a confiança.

Tentação	Por que não agora
Carrossel automático de banners	Movimento constante disputa atenção com o conteúdo, e banners de "propaganda" costumam ser ignorados (<i>banner blindness</i>).
Muitas seções ("Receitas", "Novidades", "Mais vistos"...)	Sem dados reais, seriam seções vazias ou falsas. Comece pequeno e cresça com o produto.
Animação de entrada elaborada	A Home é vista várias vezes por dia . Veja a Parte 6.5.

E a busca, as categorias e o perfil, que ainda não existem? Eles são o **núcleo** do produto e vão existir em breve. Por isso aparecem, mas **dão feedback** ao toque ("estará disponível em breve"). Um elemento que não responde ao toque parece quebrado; um que explica, não.

0.5 Estrutura de pastas

```

lib/
├── main.dart
├── app/
│   ├── celilac_app.dart          ← (alterado) usa o tema
│   ├── common/
│   │   ├── formatters.dart      ← novo: distância e decimais
│   │   └── widgets/
│   │       ├── coming_soon_view.dart ← novo: abas "em breve"
│   │       ├── gold_accent.dart
│   │       └── image_card.dart
│   ├── routes/
│   │   └── fade_route.dart      ← novo: transição com fade
│   ├── shell/
│   │   └── app_shell.dart       ← novo: barra de navegação
│   └── theme/
│       ├── app_colors.dart
│       ├── app_radius.dart      ← novo
│       ├── app_spacing.dart    ← novo
│       └── app_theme.dart       ← novo
└── features/
    ├── establishments/          ← nova feature
    │   ├── data/
    │   │   └── fake_establishment_repository.dart
    │   ├── domain/
    │   │   ├── dietary_option.dart
    │   │   ├── establishment.dart
    │   │   ├── establishment_category.dart
    │   │   └── establishment_repository.dart
    │   └── presentation/
    │       ├── establishment_category_icon.dart
    │       └── widgets/
    │           ├── dietary_badge.dart
    │           └── establishment_card.dart
    ├── home/
    │   └── presentation/
    │       ├── greeting.dart
    │       ├── pages/
    │       │   └── home_page.dart ← (reescrito)
    │       └── widgets/
    │           ├── category_shortcuts.dart
    │           ├── home_header.dart
    │           ├── nearby_states.dart
    │           ├── profile_prompt_card.dart
    │           ├── search_entry.dart
    │           └── section_header.dart
    ├── onboarding/
    └── startup/                 ← (alterado) navega para o AppShell
                                ← (alterado) navega para o AppShell

test/

```

```

├── app/
│   └── formatters_test.dart
├── features/
│   └── home/
│       ├── greeting_test.dart
│       └── home_page_test.dart

```

Por que uma feature `establishments` separada da `home`? A Home é uma **tela** que reúne coisas de vários assuntos.

"Estabelecimento" é um **assunto** do app: ele vai aparecer também em "Explorar", "Favoritos" e na busca. Se o modelo e o cartão ficassem dentro de `home/`, as outras telas teriam que importar "coisas da Home", o que não faz sentido. Regra prática:

- o que é **da tela** fica na feature da tela (`home/`: saudação, cabeçalho, cartão de perfil);
- o que é **do domínio** ganha a sua própria feature (`establishments/`: modelo, repositório, cartão de estabelecimento).

Por que `domain/` não tem subpastas? Ela reúne arquivos de tipos diferentes (um modelo, dois enums e um contrato de repositório), e é natural querer separá-los em pastas. Com apenas quatro arquivos, cujos nomes já dizem o que cada um é, as subpastas teriam um ou dois arquivos cada e só deixariam os caminhos de *import* mais longos. Pastas existem para ajudar a **encontrar** arquivos, e com quatro deles não há o que procurar. Quando o domínio crescer (algo como 8 a 10 arquivos, por exemplo com avaliações, horários de funcionamento e um segundo repositório), aí vale dividir. E a divisão deve seguir o **papel** de cada arquivo no domínio, e não o recurso da linguagem usado para escrevê-lo:

```

domain/
├── entities/                ← o que o app manipula
│   ├── establishment.dart
│   ├── establishment_category.dart
│   └── dietary_option.dart
└── repositories/           ← os contratos de acesso a dados
    └── establishment_repository.dart

```

Repare que os enums ficam **junto do modelo**, e não em uma pasta `enums/`. `EstablishmentCategory` e `DietaryOption` são os tipos de campos de `Establishment`, ou seja, fazem parte dele. Uma pasta `enums/` agruparia arquivos pela sintaxe, e não pelo significado: se um dia um desses enums virasse uma classe, o arquivo teria de mudar de pasta sem ter mudado de função. Esta é a organização usada em projetos Flutter com arquitetura em camadas, e é para ela que o CeliLac deve migrar quando o domínio pedir.

0.6 Roteiro

Parte 1	Fundação visual	tokens de espaçamento e raio, ThemeData, contraste
Parte 2	Domínio e dados	enums, modelo, contrato do repositório, dados falsos
Parte 3	Shell de navegação	NavigationBar, abas, botão "voltar", rotas
Parte 4	HomePage	slivers, saudação, busca, perfil, categorias
Parte 5	Dados assíncronos	os 4 estados, cartões, FutureBuilder, pull-to-refresh
Parte 6	Qualidade	responsividade, fonte ampliada, leitor de tela
Parte 7	Testes	unidade e widget, com repositório substituto (complementar)
Parte 8	Revisão	decisões, alternativas e próximos passos
Parte 9	No seu projeto	roteiro, checklist e desafios

Parte 1: Fundação visual

1.1 Conceito: estilo local x tema

Até aqui, cada widget definia a própria aparência:

```

FilledButton(
  style: FilledButton.styleFrom(
    backgroundColor: AppColors.navy,
    foregroundColor: Colors.white,

```

```
),
// ...
)
```

Com duas telas, isso funciona. Com dez, aparecem os mesmos problemas que os *design tokens* resolveram para as cores: repetição, divergência e dificuldade de mudar. A Home vai usar **componentes do Material** que ainda não usamos ([NavigationBar](#), [Card](#), [SnackBar](#), [SliverAppBar](#)). Em vez de estilizar cada um em cada lugar, definimos a aparência **uma vez**, no **tema**:

	Estilo local	Tema (ThemeData)
Onde fica	no próprio widget	no MaterialApp
Vale para	aquele widget	todos os widgets daquele tipo
Quando usar	exceções ("este botão é dourado")	o padrão do app

O tema é a **regra**; o estilo local é a **exceção**. Quanto mais o app cresce, mais o tema economiza.

1.2 Tokens de espaçamento e de raio

[lib/app/theme/app_spacing.dart](#)

```
abstract final class AppSpacing {
  static const double xs = 4.0;
  static const double sm = 8.0;
  static const double md = 16.0;
  static const double lg = 24.0;
  static const double xl = 32.0;
}
```

[lib/app/theme/app_radius.dart](#)

```
abstract final class AppRadius {
  static const double sm = 12.0;
  static const double md = 20.0;
  static const double lg = 40.0;
  static const double pill = 999.0;
}
```

O que significa cada sigla

Os nomes vêm dos tamanhos de roupa, em inglês, e formam uma escala do menor para o maior:

Sigla	Inglês	Leitura
xs	<i>extra small</i>	extrapequeno
sm	<i>small</i>	pequeno
md	<i>medium</i>	médio
lg	<i>large</i>	grande
xl	<i>extra large</i>	extragrande

Os valores estão em **dp** (*density-independent pixels*), a unidade do Flutter: 16 dp têm o mesmo tamanho físico aproximado em qualquer tela, seja qual for a densidade de pixels.

Espaçamentos ([AppSpacing](#)): distâncias entre elementos e margens internas.

Token	Valor	Onde é usado na Home
<code>xs</code>	4	entre o ícone e o texto do selo "Sem glúten"; entre o nome e os detalhes do cartão
<code>sm</code>	8	entre o título de uma seção e o seu conteúdo; entre o ícone e o texto da busca
<code>md</code>	16	margem interna dos cartões; espaço entre um cartão e outro
<code>lg</code>	24	margem lateral da tela; espaço entre os blocos do topo (saudação, busca, perfil)
<code>xl</code>	32	espaço antes de uma nova seção; espaço depois do último cartão

Raios (`AppRadius`): o quanto os cantos são arredondados.

Token	Valor	Onde é usado
<code>sm</code>	12	elementos pequenos dentro de outros: botão "Completar perfil", quadro do ícone no cartão, toque das categorias
<code>md</code>	20	superfícies principais: cartões, campo de busca, cartão de perfil, mensagens de erro e vazio
<code>lg</code>	40	superfícies grandes: equivale ao raio da ilustração do onboarding (<code>image_card.dart</code>), que ainda usa o número solto
<code>pill</code>	999	pontas totalmente redondas: selos de opção alimentar

Repare que `sm`, `md` e `lg` aparecem nas duas classes com **valores diferentes**: `AppSpacing.md` vale 16, e `AppRadius.md` vale 20. A sigla indica a **posição na escala** de cada classe, e não um número fixo. Por isso o nome da classe sempre acompanha a sigla no código.

Conceitos

Grade de 4 e 8. Imagine a tela coberta por uma malha de quadrados de 8×8 dp, como um papel quadriculado. A regra é que margens, espaços entre elementos e tamanhos caiam **sobre as linhas dessa malha**: 8, 16, 24, 32 dp (múltiplos de 8). O 4, meio quadrado, é o passo menor, reservado para ajustes finos entre itens muito próximos, como o ícone e o texto de um selo. É daí que vem o nome "grade de 4 e 8": **8 dp é a unidade principal e 4 dp é a meia unidade**. O Material Design e a maioria dos *design systems* seguem essa regra. Repare que todos os valores do `AppSpacing` (4, 8, 16, 24, 32) são múltiplos de 4.

Por que funciona? Com poucos valores possíveis, o olho percebe **ritmo**: os espaços "batem" entre si e as bordas se alinham. Com valores livres (13, 17, 22), a tela parece desalinhada, mesmo que ninguém saiba dizer por quê:

 Comparação entre um layout na grade de 8 dp e um com valores livres

Do lado esquerdo, todos os blocos começam na mesma margem (16 dp) e os espaços seguem um padrão: 8 dentro de um grupo, 16 entre itens e 24 antes de uma nova seção. Do lado direito, as margens variam entre 13 e 17 dp e os espaços não seguem padrão algum. A diferença é de poucos dp, mas o olho percebe o "degrau" na borda esquerda.

Por que 8, e não 5 ou 10? As densidades de tela mais comuns multiplicam os dp por 1,5, 2, 3 ou 4 para chegar aos pixels reais. 8 dp vira sempre um número inteiro de pixels (12, 16, 24, 32), e assim as bordas ficam nítidas, sem meio pixel borrado.

Nomes por tamanho, não por uso. `AppSpacing.md` em vez de `AppSpacing.cardPadding`. Um nome por uso cria um token novo para cada lugar, e voltamos aos números soltos. Com 5 tamanhos, a pergunta deixa de ser "quantos pixels?" e passa a ser "pequeno, médio ou grande?".

`AppRadius.pill = 999`. Um raio maior que metade da altura deixa as pontas totalmente arredondadas (formato de pílula), qualquer que seja a altura do elemento.

Mesma forma do `AppColors`. `abstract final class + static const`, como visto no material anterior (Parte 1.2).

No seu projeto: os valores de espaço usados na Splash e no onboarding (12, 40) ficaram fora da grade. Não é preciso refazer tudo agora: use os tokens no código **novo** e migre o antigo aos poucos, quando mexer nele.

1.3 AppTheme

`lib/app/theme/app_theme.dart`

```
import 'package:flutter/material.dart';

import 'app_colors.dart';
import 'app_radius.dart';

abstract final class AppTheme {
  static ThemeData get light {
    final colorScheme = ColorScheme.fromSeed(
      seedColor: AppColors.navy,
      primary: AppColors.navy,
      secondary: AppColors.gold,
      surface: Colors.white,
    );

    return ThemeData(
      colorScheme: colorScheme,
      scaffoldBackgroundColor: AppColors.background,
      appBarTheme: AppBarThemeData(
        backgroundColor: AppColors.background,
        foregroundColor: AppColors.navy,
        surfaceTintColor: Colors.transparent,
        shadowColor: AppColors.navy.withValues(alpha: 0.2),
        centerTitle: false,
      ),
      cardTheme: CardThemeData(
        color: Colors.white,
        elevation: 0,
        margin: EdgeInsets.zero,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(AppRadius.md),
        ),
      ),
      navigationBarTheme: NavigationBarThemeData(
        backgroundColor: Colors.white,
        indicatorColor: AppColors.gold.withValues(alpha: 0.25),
      ),
      snackBarTheme: const SnackBarThemeData(
        behavior: SnackBarBehavior.floating,
        backgroundColor: AppColors.navy,
      ),
    );
  }
}
```

Conceitos

ColorScheme: as cores por papel. O Material 3 não pergunta "qual é a cor do botão?". Ele pergunta "qual é a cor **primária**?", e cada componente sabe qual papel usar: o **FilledButton** usa **primary**, o texto sobre ele usa **onPrimary**, o fundo de cartões usa **surface**, e assim por diante. Ao definir os **papéis**, todos os componentes se ajustam juntos.

ColorScheme.fromSeed com sobrescritas. A partir de uma cor-semente, o Flutter gera uma paleta completa e harmônica (dezenas de papéis: **onPrimary**, **primaryContainer**, **outline**...). Mas a cor gerada para **primary** é uma **aproximação** da semente, não ela exatamente. Por isso informamos **primary**, **secondary** e **surface** explicitamente: a marca manda nos papéis principais, e o algoritmo preenche o resto.

scaffoldBackgroundColor. O cinza-claro da marca (**#F2F2F2**) passa a ser o fundo de **todas** as telas. Os cartões brancos (**surface: Colors.white**) se destacam sobre ele **sem precisar de sombra forte**: o contraste de cor já separa os planos.

Temas de componentes. Cada componente tem o seu "subtema":

Subtema	O que define	Por quê
appBarTheme	fundo igual ao da tela, título navy, alinhado à	a barra se integra à tela, em vez de ser uma faixa colorida

Subtema	O que define	Por quê
	esquerda	
<code>surfaceTintColor:</code> <code>Colors.transparent</code>	desliga o tingimento ao rolar	quando o conteúdo passa por baixo, o M3 eleva a barra e pode "tingi-la" com a cor primária. Na nossa <code>SliverAppBar</code> , sem esta linha, a barra ficou azul-acinzentada ao rolar (conferimos no app)
<code>shadowColor</code>	sombra navy suave ao rolar	no M3 a sombra da AppBar é transparente por padrão. Com uma cor de sombra, a elevação de rolagem aparece como uma linha suave que indica "há conteúdo atrás da barra", sem mudar a cor dela
<code>cardTheme</code>	branco, sem elevação, raio <code>md</code> , sem margem	o M3 padrão usa cartões levemente tingidos e com margem; queremos o branco da marca e controlar o espaço por fora
<code>navigationBarTheme</code>	fundo branco, indicador dourado claro	o dourado marca onde estou sem ser usado como cor de texto
<code>snackBarTheme</code>	flutuante, fundo navy	flutuando, a mensagem não cobre a barra de navegação

***ThemeData**. Repare em `AppBarThemeData`, `CardThemeData` e `NavigationBarThemeData`. Nas versões atuais do Flutter, os subtemas passados ao `ThemeData` seguem o padrão `NomeDoComponenteThemeData`. Exemplos antigos na internet usam `AppBarTheme(...)` e `CardTheme(...)`: são da API anterior.

static ThemeData get light. Um `getter` estático: `AppTheme.light` cria o tema quando é chamado. O nome `light` deixa espaço para um futuro `AppTheme.dark`.

1.4 Ligando o tema ao app

`lib/app/celilac_app.dart`

```
import 'package:flutter/material.dart';

import '../features/startup/presentation/splash_page.dart';
import 'theme/app_theme.dart';

class CeliLacApp extends StatelessWidget {
  const CeliLacApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'CeliLac',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light,
      home: const SplashPage(),
    );
  }
}
```

Atenção

- **O tema vale para o app inteiro**, inclusive Splash e onboarding. Rode o app e confira as duas telas. No CeliLac, o efeito é positivo: o fundo do onboarding passa a ser `#F2F2F2`, o que resolve parte do item 6.6 do material anterior ("fundo do onboarding igual ao da Splash").
- **Estilos locais continuam vencendo o tema**. Os botões do onboarding têm `styleFrom(background-color: AppColors.navy)` e não mudam.
- **Modo escuro**: só definimos `theme`. Sem `darkTheme`, o app fica claro mesmo com o aparelho no modo escuro. É uma escolha consciente por enquanto (veja a Parte 8).

1.5 Contraste: a matemática por trás das cores

O material anterior avisou: *"Nunca use dourado para texto"*. Agora vamos **fundamentar** essa regra e criar outras.

A WCAG (Diretrizes de Acessibilidade para Conteúdo Web, também adotadas como referência em apps) mede o **contraste** entre texto e fundo, de 1:1 (invisível) a 21:1 (preto no branco):

- **4,5:1** é o mínimo para texto comum (nível AA);
- **3:1** é o mínimo para texto grande (≥ 24 px, ou $\geq 18,7$ px em negrito) e para ícones e bordas que transmitem informação.

Calculamos o contraste das combinações da marca:

Texto	Fundo	Contraste	Uso permitido
navy	branco	11,3:1	qualquer texto
navy	#F2F2F2	10,1:1	qualquer texto
navy 80%	#F2F2F2	5,7:1	texto secundário
navy 70%	#F2F2F2	4,35:1	insuficiente: abaixo de 4,5 para texto comum
navy 60%	#F2F2F2	3,4:1	só texto grande
dourado	branco	2,2:1	nunca texto ; só decoração
navy	dourado	5,0:1	texto navy sobre botão dourado
branco	navy	11,3:1	texto branco sobre cartão navy
branco 85%	navy	8,7:1	texto secundário sobre navy

Dessa tabela saem as **regras da Home**:

1. **Texto secundário = navy 80%** (`AppColors.navy.withValues(alpha: 0.8)`). No onboarding usamos 70%, que fica em 4,6:1 sobre branco, mas cai para 4,35:1 sobre o cinza da tela. A Home tem fundo cinza, então subimos para 80%.
2. **Dourado é fundo ou decoração**, nunca cor de texto: barrinha, indicador da navegação, fundo de selos e de botões.
3. **Sobre dourado, texto navy**. Sobre navy, texto branco.

Para conferir outras combinações: qualquer "WCAG contrast checker" on-line recebe as duas cores em hexadecimal e calcula a razão. Lembre que uma cor com transparência deve ser medida **já misturada** com o fundo.

No seu projeto: monte essa tabela com as cores da sua marca **antes** de escolher as cores dos textos. É comum descobrir que a cor "bonita" da marca não serve para texto.

Execute: com o tema aplicado, abra o app e percorra Splash e onboarding. Nada deve ter "quebrado"; o fundo do onboarding agora é cinza-claro.

Parte 2: Domínio e dados

2.1 Conceito: a tela não inventa os dados

A forma mais rápida de fazer a lista "Perto de você" seria escrever os cartões direto na tela:

```
// Errado: dados misturados com a interface
Column(children: [
  Text('Café Aconchego'),
  Text('Centro · 350 m'),
  // ...
])
```

Funciona até o dia em que os dados vierem de uma API. Nesse dia, **a tela inteira** precisa ser reescrita. A abordagem profissional separa três perguntas:

Pergunta	Camada	Arquivo
O que é um estabelecimento?	domain	establishment.dart
Como obtenho estabelecimentos? (contrato)	domain	establishment_repository.dart
De onde eles vêm hoje? (implementação)	data	fake_establishment_repository.dart
Como eles aparecem?	presentation	establishment_card.dart

Quando a API existir, só a última linha de **data** muda: criamos um **ApiEstablishmentRepository**, e a tela não percebe a troca.

2.2 Enums com valor: opções alimentares e categorias

O problema: valores que só podem ser alguns poucos

Um estabelecimento do CeliLac tem duas informações que **não aceitam qualquer valor**:

- a **categoria**: restaurante, padaria, café ou mercado;
- as **opções alimentares**: sem glúten, sem lactose.

Não existe a categoria "xyz", nem a opção "talvez sem glúten". São **conjuntos fechados**: sabemos de antemão todos os valores possíveis, e eles mudam raramente (só quando o produto decide, e alguém altera o código).

A primeira ideia costuma ser guardar esses valores como texto:

```
// Com String: funciona, mas é frágil
final category = 'Padaria';

if (category == 'padaria') { // nunca é verdadeiro: P maiúsculo x minúsculo
  // ...
}
```

Nada impede que 'Padaria', 'padaria', 'Padaría' e 'bakery' apareçam em lugares diferentes do código. Para o Dart, todos são textos válidos, e o erro só aparece **com o app rodando**, quando um filtro não encontra nada ou uma tela mostra o rótulo errado. É um *bug* silencioso: nada avisa.

O que é um **enum**

Um **enum** (de *enumeration*, enumeração) é um **tipo** cujos valores possíveis são **listados um a um** na sua declaração. Nenhum outro valor existe. É o equivalente, no código, a um campo de formulário de múltipla escolha: em vez de uma caixa de texto livre, só as opções da lista.

A forma mais simples:

```
enum DietaryOption { glutenFree, lactoseFree }
```

Isto cria o tipo **DietaryOption** com **exatamente** dois valores. Para usá-los, escreve-se o nome do tipo, um ponto e o nome do valor:

```
final option = DietaryOption.glutenFree;

if (option == DietaryOption.glutenFree) {
  // ...
}
```

Agora, um erro de digitação **não compila**: **DietaryOption.glutenfree** (com "f" minúsculo) ou **DietaryOption.vegan** fazem o editor sublinhar a linha em vermelho antes mesmo de rodar o app. O erro sai da tela do usuário e vai para a tela do desenvolvedor.

O que todo enum oferece

Sem escrever nada a mais, todo enum do Dart já vem com:

Recurso	Exemplo	Resultado
a lista de todos os valores, na ordem da declaração	<code>DietaryOption.values</code>	<code>[DietaryOption.glutenFree, DietaryOption.lactoseFree]</code>
o nome do valor, como texto	<code>DietaryOption.glutenFree.name</code>	<code>'glutenFree'</code>
a posição na lista, a partir de 0	<code>DietaryOption.lactoseFree.index</code>	<code>1</code>
busca pelo nome	<code>DietaryOption.values.byName('lactoseFree')</code>	<code>DietaryOption.lactoseFree</code>
comparação com <code>==</code>	<code>option == DietaryOption.glutenFree</code>	<code>true</code> ou <code>false</code>

E, a partir do Dart 3, o **switch verifica se todos os valores foram tratados**. Se um valor novo for acrescentado ao enum e algum **switch** não o tratar, o app não compila (veja o **switch** da Parte 4.6). É o compilador avisando todos os lugares que precisam ser atualizados.

Atenção: não use `.name` como texto da interface. Ele devolve o identificador do código (`'glutenFree'`), não algo que o usuário deva ler. Para isso existe o `label`, a seguir.

Enum com campos (*enhanced enum*)

Falta uma coisa: o texto que o usuário lê. Poderíamos espalhar pelo código um **if** ou um **switch** que converte `glutenFree` em `'Sem glúten'`, mas aí o texto ficaria longe do valor, e cada tela poderia escrevê-lo de um jeito. Desde o Dart 2.17, um enum pode ter **campos e um construtor**, como uma classe. Assim, cada valor carrega os seus próprios dados:

`lib/features/establishments/domain/dietary_option.dart`

```
enum DietaryOption {
  glutenFree('Sem glúten'),
  lactoseFree('Sem lactose');

  const DietaryOption(this.label);

  final String label;
}
```

Linha a linha:

Trecho	O que faz
<code>glutenFree('Sem glúten'),</code>	declara o valor <code>glutenFree</code> e passa <code>'Sem glúten'</code> ao construtor, como em <code>DietaryOption('Sem glúten')</code>
<code>lactoseFree('Sem lactose');</code>	o último valor termina com ponto e vírgula , e não com vírgula: ele separa a lista de valores do resto da declaração. Esquecê-lo é o erro mais comum ao escrever o primeiro enum com campos
<code>const DietaryOption(this.label);</code>	o construtor. Em um enum ele precisa ser const , porque os valores são criados uma única vez, quando o programa compila
<code>final String label;</code>	o campo. Precisa ser final : o valor de um enum nunca muda

O uso fica direto: `DietaryOption.glutenFree.label` devolve `'Sem glúten'`. O texto mora **junto do valor, em um único lugar**. Se o produto decidir trocar "Sem glúten" por "Livre de glúten", muda uma linha, e todas as telas acompanham.

O mesmo padrão vale para as categorias:

`lib/features/establishments/domain/establishment_category.dart`

```
enum EstablishmentCategory {
  restaurant('Restaurante'),
  bakery('Padaria'),
  cafe('Café'),
  market('Mercado');

  const EstablishmentCategory(this.label);

  final String label;
}
```

Conceitos

EstablishmentCategory.values **desenha a Home**. A Home usa essa lista para criar os atalhos de categoria (Parte 4.6): um atalho para cada valor, na ordem da declaração. Uma categoria nova no enum aparece na Home **sem mexer na Home**. Para mudar a ordem dos atalhos, basta mudar a ordem das linhas no enum.

Os nomes em inglês, os textos em português. Os identificadores (`glutenFree`) seguem a convenção do código; o que o usuário lê ('Sem glúten') segue o idioma do app.

Quando usar enum, e quando não usar. Use enum quando **todos** os valores forem conhecidos ao escrever o código e só mudarem com uma nova versão do app: categorias, status de um pedido ("pendente", "pago", "entregue"), dias da semana. **Não** use quando os valores vierem de fora e puderem crescer sem atualizar o app: os próprios estabelecimentos, por exemplo, vêm de um repositório e por isso são uma **classe** (Parte 2.3), não um enum.

E quando a API devolver texto? Um servidor envia `"bakery"`, e não `EstablishmentCategory.bakery`. A conversão acontece **uma vez**, na camada `data`, com `EstablishmentCategory.values.byName('bakery')`, e daí para dentro o app só trabalha com o enum. Se o servidor enviar um nome desconhecido, `byName` lança um erro, e é ali, na entrada, que ele deve ser tratado.

E o ícone de cada categoria? Ele **não** fica aqui. `IconData` vem do Flutter, e a camada `domain` não depende de Flutter (mesmo critério do `OnboardingItem`, no material anterior). O ícone fica em `presentation` (Parte 4.6).

2.3 O modelo Establishment

`lib/features/establishments/domain/establishment.dart`

```
import 'dietary_option.dart';
import 'establishment_category.dart';

class Establishment {
  const Establishment({
    required this.id,
    required this.name,
    required this.category,
    required this.neighborhood,
    required this.distanceInKm,
    required this.rating,
    required this.dietaryOptions,
  });

  final String id;
  final String name;
  final EstablishmentCategory category;
  final String neighborhood;
  final double distanceInKm;
  final double rating;
  final Set<DietaryOption> dietaryOptions;
}
```

Conceitos

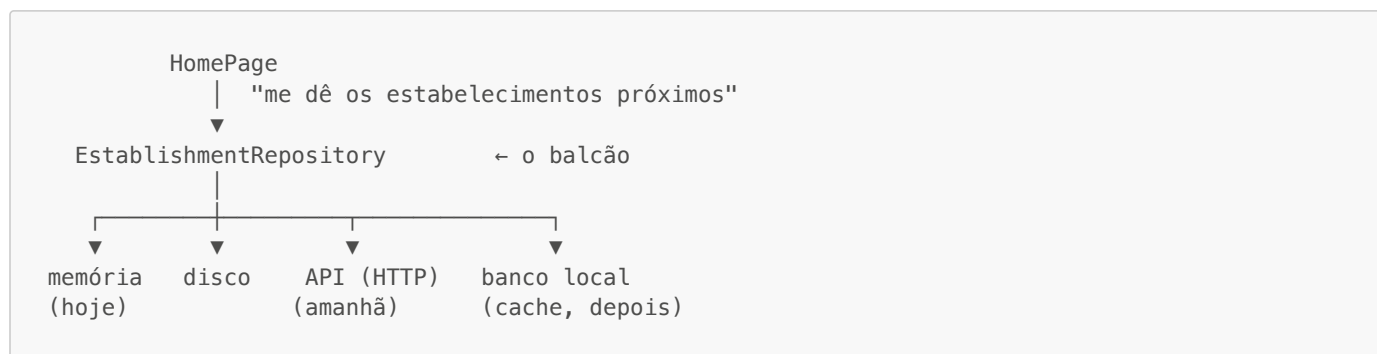
- **Imutável e `const`**, como o `OnboardingItem`.
- **`id`**: mesmo sem uso hoje, todo dado que virá de um servidor precisa de um identificador. Favoritar, abrir o detalhe e comparar dois itens dependem dele.
- **`distanceInKm`**: a **unidade no nome** evita a dúvida "isso é metro ou quilômetro?". O modelo guarda o **número**; a formatação ("350 m", "1,2 km") é responsabilidade da apresentação (Parte 5.2).
- **`Set<DietaryOption>`** em vez de `List`: um estabelecimento não é "sem glúten" duas vezes. O `Set` **garante** que não há repetição, e `contains` é a operação natural ("ele atende sem lactose?").
- **Em vez de dois booleanos** (`isGlutenFree`, `isLactoseFree`): com o `Set`, uma terceira opção (por exemplo, "vegano") é só um novo valor no enum, sem mudar o modelo.

2.4 O contrato: `EstablishmentRepository`

O que é um repositório

Pense no balcão de uma biblioteca. Você pede um livro pelo título, e o atendente o entrega. Você não sabe se ele estava na estante do térreo, no depósito do subsolo ou se veio emprestado de outra unidade, e nem precisa saber. Se amanhã a biblioteca reorganizar o depósito, o seu jeito de pedir continua o mesmo.

O **repositório** é esse balcão, dentro do app. É o objeto responsável por **obter** (e, quando for o caso, **salvar**) os dados de um assunto: estabelecimentos, usuários, pedidos. Quem precisa dos dados (uma tela, por exemplo) **pede ao repositório** e recebe objetos do domínio prontos para usar, como `List<Establishment>`. Quem pede não sabe de onde os dados vieram:



O nome vem de um **padrão de projeto** (*Repository pattern*) descrito por Martin Fowler e por Eric Evans no início dos anos 2000, e hoje usado em praticamente todas as plataformas (Android, iOS, web, *back-end*).

O que é responsabilidade do repositório:

- buscar os dados na origem (memória, arquivo, API, banco);
- converter o formato da origem (por exemplo, JSON) em objetos do domínio (`Establishment`);
- decidir estratégias como *cache* ("já busquei há 1 minuto, devolvo o que tenho").

O que não é:

- decidir como os dados aparecem na tela (isso é da *presentation*);
- formatar textos para o usuário, como "350 m" (Parte 5.2).

Os métodos falam a língua do app, e não da tecnologia. O método se chama `fetchNearby()` ("buscar os próximos"), e não `getFromHttp()` ou `selectFromTable()`. O nome descreve **o que** a tela quer; o **como** fica escondido dentro da implementação.

O que se ganha com isso:

Benefício	No CeliLac
trocar a origem dos dados sem mexer nas telas	hoje os dados são falsos; quando a API existir, só nasce uma nova implementação (Parte 2.5)
testar a tela com dados controlados	os testes entregam à Home um repositório que devolve sucesso, vazio ou erro sob encomenda (Parte 7)
um único lugar para cada regra de acesso a dados	<i>cache</i> , novas tentativas e tratamento de erros de rede não se espalham pelas telas

Para quem for ler a documentação do Flutter: o guia oficial de arquitetura (docs.flutter.dev/app-architecture) também usa repositórios, e separa ainda os *services*: classes que só conversam com a API ou com o disco, usadas pelo repositório. No CeliLac, que ainda não tem API, o repositório faz os dois papéis. A separação pode vir junto com a API.

O contrato em código

Para a tela "pedir sem saber de onde vem", ela precisa depender de uma **descrição** do repositório, e não de uma implementação concreta. Essa descrição é o **contrato**: a lista de métodos que qualquer repositório de estabelecimentos precisa oferecer, sem dizer como.

`lib/features/establishments/domain/establishment_repository.dart`

```
import 'establishment.dart';

abstract interface class EstablishmentRepository {
  Future<List<Establishment>> fetchNearby();
}
```

O método não tem corpo (`{ ... }`): termina em `;`. Ele diz **o que** existe (um método `fetchNearby` que devolve, no futuro, uma lista de estabelecimentos), mas não **como** funciona. O "como" fica com cada implementação.

Interfaces em Dart: antes e depois do Dart 3

É comum ouvir que "o Dart não tem interface", e até o Dart 3 isso era **meio verdade**. Vale entender as duas épocas, porque você vai encontrar código e tutoriais de ambas.

Até o Dart 2: interfaces implícitas. O Dart nunca teve uma palavra `interface` separada, como Java e C# têm. Em vez disso, **toda classe já define automaticamente uma interface**: o conjunto dos seus métodos e campos públicos. Qualquer classe podia ser usada de dois jeitos:

```
class Animal {
  void speak() => print('...');
}

class Dog extends Animal {} // herda: ganha o speak() pronto

class Robot implements Animal { // implementa: promete ter speak(),
  @override                    // mas escreve o próprio corpo
  void speak() => print('bip');
}
```

	<code>extends</code> (herdar)	<code>implements</code> (implementar)
o que recebe	o código da classe-mãe	só a obrigação de ter os mesmos membros
quantas classes	só uma	várias (<code>implements A, B</code>)
corpo dos métodos	herdado (pode sobrescrever)	precisa escrever todos

Por isso, para criar um contrato, a convenção era escrever uma `abstract class` só com métodos sem corpo e **combinar** que ela seria usada com `implements`. Mas era só uma combinação: nada impedia alguém de usar `extends`, e a linguagem não tinha como expressar "esta classe é só um contrato".

No Dart 3 (maio de 2023): modificadores de classe. O Dart 3 acrescentou palavras que se colocam **antes de `class`** para controlar como ela pode ser usada: `interface`, `base`, `final`, `sealed` e `mixin`. Repare: `interface` não é um tipo novo, separado de `class`; é um **modificador** de uma classe. Por isso se escreve `interface class`, e não só `interface`.

`interface class` significa: **fora do arquivo onde foi declarada, esta classe só pode ser implementada (`implements`), nunca herdada (`extends`)**. Combinada com `abstract`, temos o contrato completo:

Palavra	Efeito	Erro se desrespeitado
<code>abstract</code>	não pode ser instanciada; permite métodos sem corpo	<code>EstablishmentRepository()</code> → "Abstract classes can't be instantiated"
<code>interface</code>	fora do seu arquivo, só pode ser usada com <code>implements</code>	<code>extends EstablishmentRepository</code> → "can't be extended outside of its library because it's an interface class"
<code>abstract interface</code>	as duas coisas: é só um contrato	—

Testamos essas regras no Dart 3.13 deste projeto. Três detalhes que costumam surpreender:

- **"Fora do seu arquivo"**. Para o Dart, cada arquivo `.dart` é uma **biblioteca** (*library*), e os modificadores só valem **fora** dela. Dentro de `establishment_repository.dart`, um `extends` ainda compilaria. Na prática, como o contrato fica sozinho no seu arquivo, toda implementação está em outro arquivo, e a regra vale.
- **interface class sem abstract pode ser instanciada**. `interface` só proíbe herdar; quem proíbe criar objetos é `abstract`. Para um contrato, os dois juntos.
- **As interfaces implícitas continuam existindo**. No Dart 3, qualquer classe comum ainda pode ser implementada com `implements`. Os modificadores servem para **restringir** e **deixar a intenção explícita**, e não para criar algo que antes era impossível.

Por que usar `abstract interface class` no CeliLac, se uma `abstract class` funcionaria? Porque a declaração passa a **dizer** a intenção, e o compilador passa a **garantir** essa intenção. Quem abre o arquivo lê "isto é um contrato", e não precisa adivinhar se deve herdar ou implementar.

O `FakeEstablishmentRepository` (Parte 2.5) cumpre o contrato assim:

```
class FakeEstablishmentRepository implements EstablishmentRepository {
  @override
  Future<List<Establishment>> fetchNearby() async { /* ... */ }
}
```

Se ele esquecer o método, o erro aparece na hora: "Missing concrete implementation of 'EstablishmentRepository.fetchNearby'".

Conceitos

`Future<List<Establishment>>`. Mesmo que os dados de hoje estejam na memória, o contrato já é **assíncrono**. Buscar dados de verdade sempre leva tempo, e mudar a assinatura depois obrigaria a mudar todas as telas.

Por que o contrato fica em `domain`? Porque quem **usa** o contrato (a tela) e quem **cumpr**e o contrato (`data`) dependem dele, e ele não depende de nenhum dos dois.

```
presentation —usa—> domain (contrato) <—implementa— data
  HomePage           EstablishmentRepository   FakeEstablishmentRepository
```

2.5 A implementação falsa: `FakeEstablishmentRepository`

`lib/features/establishments/data/fake_establishment_repository.dart`

```
import '../domain/dietary_option.dart';
import '../domain/establishment.dart';
import '../domain/establishment_category.dart';
import '../domain/establishment_repository.dart';

class FakeEstablishmentRepository implements EstablishmentRepository {
  const FakeEstablishmentRepository({
    this.delay = const Duration(milliseconds: 800),
  });
```

```

final Duration delay;

static const List<Establishment> _establishments = [
  Establishment(
    id: '1',
    name: 'Café Aconchego',
    category: EstablishmentCategory.cafe,
    neighborhood: 'Centro',
    distanceInKm: 0.35,
    rating: 4.8,
    dietaryOptions: {DietaryOption.glutenFree, DietaryOption.lactoseFree},
  ),
  Establishment(
    id: '2',
    name: 'Padaria Leve Grão',
    category: EstablishmentCategory.bakery,
    neighborhood: 'Jardim das Flores',
    distanceInKm: 1.2,
    rating: 4.6,
    dietaryOptions: {DietaryOption.glutenFree},
  ),
  Establishment(
    id: '3',
    name: 'Bistrô Bem-Estar',
    category: EstablishmentCategory.restaurant,
    neighborhood: 'Bela Vista',
    distanceInKm: 2.4,
    rating: 4.7,
    dietaryOptions: {DietaryOption.glutenFree, DietaryOption.lactoseFree},
  ),
  Establishment(
    id: '4',
    name: 'Empório Natural',
    category: EstablishmentCategory.market,
    neighborhood: 'Vila Nova',
    distanceInKm: 3.1,
    rating: 4.5,
    dietaryOptions: {DietaryOption.lactoseFree},
  ),
];

@override
Future<List<Establishment>> fetchNearby() async {
  await Future<void>.delayed(delay);
  return _establishments;
}
}

```

Conceitos

Por que "falso" é profissional? Times de verdade constroem a interface **antes** de a API ficar pronta. Uma implementação falsa (*fake*) permite desenvolver, demonstrar e testar a tela hoje. Ela é uma **implementação legítima do contrato**, e não um improviso.

Future.delayed simula a rede. Sem o atraso, os dados chegariam "instantaneamente" e nunca veríamos o estado de carregamento. Com 800 ms, a tela é exercitada **como será com a API real**. Esse é o motivo de existir o parâmetro *delay*: os testes podem passar *Duration.zero*.

implements + @override. O compilador verifica que *fetchNearby* existe com a assinatura do contrato. Se o contrato mudar, o *fake* deixa de compilar, e você é avisado.

Dados plausíveis e fictícios. Nomes inventados (sem marcas reais), variedade de categorias, distâncias abaixo e acima de 1 km, estabelecimentos com uma ou duas opções alimentares. Dados variados **revelam problemas de layout** que dados repetidos escondem: nome longo, bairro longo, dois selos.

Ordenados por distância. A seção se chama "Perto de você"; a ordem dos dados confirma o título.

No seu projeto: crie o *fake* com **4 a 6 itens** variados, incluindo pelo menos um com o texto mais longo que você imagina existir. Isso testa o seu layout desde o primeiro dia.

Para ver o estado de erro (Parte 5), troque temporariamente o corpo de `fetchNearby` por:

```
await Future<void>.delayed(delay);
throw Exception('Falha simulada');
```

E para ver o estado vazio, devolva `const []`. **Desfaça** depois de testar.

Parte 3: Shell de navegação

3.1 Conceito: a Home não é o app

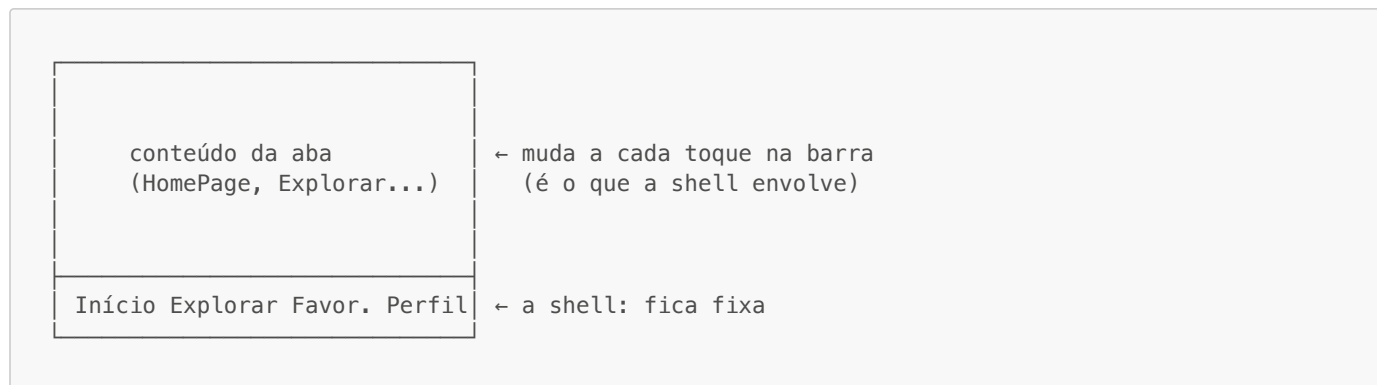
Até agora, "ir para a Home" significava abrir **uma tela**. Mas um app com várias áreas (início, explorar, favoritos, perfil) se comporta de outro jeito. Abra qualquer app que você usa todo dia, de banco, de música ou de entregas, e toque nos itens da barra inferior: o **meio** da tela muda por completo, e a **barra continua no mesmo lugar**, sem piscar e sem animação de troca de tela.

Esse é o comportamento que vamos construir. A parte que fica parada é a **shell**.

O que é a shell

Shell é uma palavra em inglês que significa **casca** (como a de um ovo ou de um caracol). Em interfaces, ela designa a **estrutura fixa que envolve o conteúdo variável** de um app: os elementos que aparecem em todas as áreas e não mudam quando o usuário passa de uma área para outra.

Uma comparação: a shell é a **moldura de um porta-retratos digital**. As fotos (o conteúdo) trocam; a moldura (a shell) permanece. Ou o **painel de um carro**: você troca a estação de rádio, mas o painel continua o mesmo.



O que pertence à shell: só o que é **comum a todas as áreas**. No CeliLac, é a barra de navegação inferior. Em outros apps, também pode ser um menu lateral (*drawer*) ou um botão de ação que aparece em todas as abas.

O que não pertence: o conteúdo de cada área. A barra superior com o logo, por exemplo, é da **HomePage**, e não da shell: as outras abas podem querer outro título.

No Flutter, a shell não é um widget especial. Não existe um `Shell` pronto na biblioteca. Ela é uma tela comum (um `StatefulWidget` com um `Scaffold`) que guarda **qual área está ativa** e mostra essa área no **body**. O nome `AppShell` é uma convenção, e não um componente do framework.

Por que não fazer cada aba como uma tela separada, com `Navigator.push`? Daria para colocar uma `NavigationBar` em cada tela e navegar entre elas. Mas:

Sem shell (uma tela por aba)

a barra é repetida em todas as telas, e cada cópia precisa ser mantida igual

Com shell

a barra existe **uma vez**

Sem shell (uma tela por aba)	Com shell
trocar de aba é uma navegação : há animação de transição, e a tela "chega" por cima	trocar de aba é só trocar o conteúdo : a barra não se mexe
cada troca empilha uma tela; o botão "voltar" percorre todo o histórico de abas	o "voltar" é controlado em um só lugar (veja o PopScope , na Parte 3.4)
a tela anterior pode ser destruída, e a rolagem e os dados se perdem	as abas podem ser mantidas vivas (veja o IndexedStack , na Parte 3.4)

Você vai reencontrar o termo. Ele vem do desenvolvimento web, onde *app shell* designa a parte mínima da interface (cabeçalho, menu) que carrega primeiro e fica fixa enquanto o conteúdo chega. No Flutter, o pacote de rotas [go_router](#), muito usado em apps maiores, tem [ShellRoute](#) e [StatefulShellRoute](#): exatamente esta ideia, aplicada às rotas. O que construímos aqui à mão é o mesmo conceito, na sua forma mais simples.

A estrutura do [AppShell](#) no CeliLac:

```
AppShell (Scaffold)
├── body: a aba selecionada
│   ├── 0: HomePage
│   ├── 1: Explorar (em breve)
│   ├── 2: Favoritos (em breve)
│   └── 3: Perfil (em breve)
└── bottomNavigationBar: NavigationBar ← fixa
```

Separar shell e Home traz duas vantagens:

- a [HomePage](#) cuida **só do seu conteúdo** (não sabe que existe uma barra embaixo);
- Splash e onboarding navegam para o **app** ([AppShell](#)), e não para uma aba específica.

Por que estas quatro abas?

Aba	Promessa	Justificativa
Início	todas	o resumo do dia; o ponto de partida
Explorar	Descoberta	busca e filtros completos, mapa
Favoritos	Descoberta	quem acha um lugar seguro volta a ele; é um hábito comum de quem tem restrição alimentar
Perfil	Personalização	o perfil alimentar e as preferências

Regras das diretrizes do Material Design para a barra inferior:

- **3 a 5 destinos.** Menos que 3: use abas no topo ou nem use barra. Mais que 5: os rótulos ficam apertados e as escolhas, difíceis.
- **Destinos de mesmo nível e sempre acessíveis:** não use a barra para ações ("Adicionar", "Sair").
- **Ícone + rótulo** sempre. Ícones sozinhos são ambíguos (um coração é "favoritos" ou "saúde"?).
- **Ícone contornado** quando inativo, **preenchido** quando ativo: o estado não depende só de cor.

3.2 Uma rota com fade reutilizável

O material anterior (Parte 6.5) recomendou extrair a transição com *fade*. Agora ela será usada em **dois** lugares (Splash e onboarding), então chegou a hora.

[lib/app/routes/fade_route.dart](#)

```
import 'package:flutter/material.dart';

PageRouteBuilder<void> fadeRoute(Widget page) {
  return PageRouteBuilder<void>(
    transitionDuration: const Duration(milliseconds: 500),
```

```

    pageBuilder: (_, __, _) => page,
    transitionsBuilder: (_, animation, __, child) =>
      FadeTransition(opacity: animation, child: child),
  );
}

```

Uma função, e não uma classe. Não há estado nem configuração: uma função que recebe a página e devolve a rota é a forma mais simples que resolve o problema.

3.3 Abas "em breve": **ComingSoonView**

lib/app/common/widgets/coming_soon_view.dart

```

import 'package:flutter/material.dart';

import '../..../theme/app_colors.dart';
import '../..../theme/app_spacing.dart';

class ComingSoonView extends StatelessWidget {
  const ComingSoonView({super.key, required this.title, required this.icon});

  final String title;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;

    return SafeArea(
      child: Center(
        child: Padding(
          padding: const EdgeInsets.all(AppSpacing.xl),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              Icon(icon, size: 48, color: AppColors.navy),
              const SizedBox(height: AppSpacing.md),
              Text(
                title,
                style: textTheme.titleLarge?.copyWith(
                  color: AppColors.navy,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: AppSpacing.sm),
              Text(
                'Estamos preparando esta área para você.',
                style: textTheme.bodyMedium?.copyWith(
                  color: AppColors.navy.withValues(alpha: 0.8),
                ),
                textAlign: TextAlign.center,
              ),
            ],
          ),
        ),
      ),
    );
  }
}

```

Por que ter abas que ainda não existem? A barra de navegação comunica a **estrutura** do produto. Ao vê-la, o usuário entende o que o app faz e o que está por vir. Uma aba "em breve" **honesta** é melhor do que (a) uma barra que muda de tamanho a cada versão ou (b) uma aba que abre uma tela em branco.

`mainAxisSize: MainAxisSize.min`. A `Column` ocupa só a altura dos filhos, e o `Center` consegue centralizá-la na tela.

3.4 AppShell

lib/app/shell/app_shell.dart

```
import 'package:flutter/material.dart';

import '../features/establishments/data/fake_establishment_repository.dart';
import '../features/home/presentation/pages/home_page.dart';
import '../common/widgets/coming_soon_view.dart';

class AppShell extends StatefulWidget {
  const AppShell({super.key});

  @override
  State<AppShell> createState() => _AppShellState();
}

class _AppShellState extends State<AppShell> {
  int _selectedIndex = 0;

  static const List<Widget> _tabs = [
    HomePage(repository: FakeEstablishmentRepository()),
    ComingSoonView(title: 'Explorar', icon: Icons.explore_outlined),
    ComingSoonView(title: 'Favoritos', icon: Icons.favorite_outline),
    ComingSoonView(title: 'Perfil', icon: Icons.person_outline),
  ];

  void _onDestinationSelected(int index) {
    setState(() => _selectedIndex = index);
  }

  @override
  Widget build(BuildContext context) {
    return PopScope(
      canPop: _selectedIndex == 0,
      onPopInvokedWithResult: (didPop, _) {
        if (!didPop) {
          _onDestinationSelected(0);
        }
      },
      child: Scaffold(
        body: IndexedStack(index: _selectedIndex, children: _tabs),
        bottomNavigationBar: NavigationBar(
          selectedIndex: _selectedIndex,
          onDestinationSelected: _onDestinationSelected,
          destinations: const [
            NavigationDestination(
              icon: Icon(Icons.home_outlined),
              selectedIcon: Icon(Icons.home),
              label: 'Início',
            ),
            NavigationDestination(
              icon: Icon(Icons.explore_outlined),
              selectedIcon: Icon(Icons.explore),
              label: 'Explorar',
            ),
            NavigationDestination(
              icon: Icon(Icons.favorite_outline),
              selectedIcon: Icon(Icons.favorite),
              label: 'Favoritos',
            ),
            NavigationDestination(
              icon: Icon(Icons.person_outline),
              selectedIcon: Icon(Icons.person),
            ),
          ],
        ),
      ),
    );
  }
}
```

```

        label: 'Perfil',
      ),
    ],
  ),
);
}
}

```

Conceitos

NavigationBar × **BottomNavigationBar**. Os dois desenhavam uma barra inferior. O **BottomNavigationBar** é o componente do **Material 2**; o **NavigationBar** é o do **Material 3**, com indicador em pílula atrás do ícone ativo, altura maior (área de toque mais confortável) e cores vindas do **ColorScheme**. Em projetos novos, use o **NavigationBar**.

Estado mínimo: só o índice. O shell guarda **qual aba está selecionada** e nada mais. **selectedIndex** diz à barra qual destacar; **onDestinationSelected** avisa quando o usuário toca em outra. O mesmo ciclo do onboarding: evento → **setState** → **build**.

IndexedStack em vez de trocar o filho. A alternativa ingênua seria **body: _tabs[_selectedIndex]**. Ela funciona, mas **destrói** a aba anterior ao trocar: ao voltar para o Início, a lista recarregaria e a rolagem voltaria ao topo. O **IndexedStack** mantém **todas** as abas vivas e mostra só uma. O usuário volta exatamente para onde estava.

Atenção: há um custo. Todas as abas são construídas logo na abertura. Com abas leves como as de hoje, não há problema. Quando uma aba ficar pesada (mapa, câmera), considere construí-la só na primeira visita.

static const List<Widget> _tabs. As abas são **const** porque todos os construtores envolvidos são **const** (inclusive o do **FakeEstablishmentRepository**). O Flutter cria essa lista **uma única vez**.

O shell "monta" a Home. É o **AppShell** que decide **qual** repositório a Home usa: **HomePage(repository: FakeEstablishmentRepository())**. A Home só conhece o contrato. Esse ponto onde as peças concretas são escolhidas e conectadas é chamado de **raiz de composição** (*composition root*). Quando a API existir, a troca é **nesta linha**.

PopScope: o botão "voltar" do Android. Sem ele, quem está na aba "Perfil" e aperta "voltar" **fecha o app**, o que surpreende. O comportamento esperado é voltar para o Início e, **só de lá**, sair do app:

Perfil —voltar→ Início —voltar→ sai do app

- **canPop: _selectedIndex == 0:** só permite fechar a tela quando estamos no Início;
- **onPopInvokedWithResult:** é chamado em toda tentativa de voltar. Se o "voltar" foi bloqueado (**didPop == false**), vamos para o Início.

PopScope substituiu o antigo **WillPopScope** (descontinuado), e é compatível com o "voltar preditivo" do Android 14+.

3.5 Splash e onboarding passam a abrir o AppShell

Onboarding (**onboarding_page.dart**): troque os **imports** e o destino de **_finishOnboarding**.

```

// remova:
import '../home/presentation/pages/home_page.dart';

// adicione:
import '../routes/fade_route.dart';
import '../shell/app_shell.dart';

```

Substitua o método **_finishOnboarding** **inteiro**, da assinatura até a chave **}** que o fecha:

```

Future<void> _finishOnboarding() async {
  final storage = OnboardingStorage();
  await storage.markAsCompleted();
}

```

```

    if (!mounted) {
      return;
    }

    Navigator.of(context).pushReplacement(fadeRoute(const AppShell()));
  }

```

Cuidado com chaves sobrando. Se você trocar só a linha do `Navigator` e colar também a `}` do trecho acima, o método ganha uma chave a mais: ela fecha a **classe** antes da hora. O `flutter analyze` mostra então uma cascata de erros enganosos, como *"Missing concrete implementation of 'State.build'", "Undefined name '_currentPage'"* e *"The function 'setState' isn't defined"*. Todos somem quando a chave extra é removida.

Splash (`splash_page.dart`): ative a **decisão de rota final** (material anterior, Parte 4.5), agora com o `AppShell`, e use `fadeRoute` em `_goTo`.

```

// remova:
import '../..//home/presentation/pages/home_page.dart';

// adicione:
import '../..//app/routes/fade_route.dart';
import '../..//app/shell/app_shell.dart';

// mantenha (ou adicione de volta, se tiver removido):
import '../..//onboarding/data/onboarding_storage.dart';

```

Atenção: enquanto a decisão de rota estava comentada, o `flutter analyze` acusava o `import` de `onboarding_storage.dart` como "não usado", e é comum removê-lo. Agora ele volta a ser necessário: sem ele, `OnboardingStorage` fica indefinido.

Substitua os métodos `_initialize` e `_goTo` **inteiros**, apagando também o bloco de código comentado que ficava no fim de `_initialize`:

```

Future<void> _initialize() async {
  await _introController.forward();
  if (!mounted) {
    return;
  }

  await _progressController.forward();
  if (!mounted) {
    return;
  }

  final completed = await OnboardingStorage().isCompleted();
  if (!mounted) {
    return;
  }

  _goTo(completed ? const AppShell() : const OnboardingPage());
}

void _goTo(Widget page) {
  Navigator.of(context).pushReplacement(fadeRoute(page));
}

```

Conceitos

- A linha `final completed = ...` é obrigatória. É ela que declara a variável usada em `completed ? const AppShell() : const OnboardingPage()`. Sem ela, o erro é *"Undefined name 'completed'"*.

- **As duas entradas do app chegam ao mesmo lugar**, com a **mesma transição**. Isso resolve o item 6.5 do material anterior.
- **fade_route.dart** fica em **lib/app/routes/**, e não dentro de uma feature: a transição é usada pelo app inteiro, não é assunto de estabelecimentos nem de onboarding.
- **O código comentado da Splash some**. Os avisos de *imports* sem uso que o **flutter analyze** mostrava também desaparecem.

Execute:

- primeira execução: Splash → onboarding → "Começar" → barra inferior com quatro abas;
- toque em cada aba: as três últimas mostram "em breve";
- na aba "Perfil", aperte "voltar" (Android): o app vai para o Início; aperte de novo: o app fecha;
- reabra o app: Splash → **direto** para o shell.

Dica: para rever o onboarding durante o desenvolvimento, desinstale o app ou limpe os dados dele (material anterior, Parte 5.3).

Parte 4: HomePage: estrutura e seções

4.1 Conceito: por que *slivers*?

A Home é uma tela **rolável** com conteúdos de naturezas diferentes: uma barra no topo, blocos fixos (saudação, busca, cartão) e uma **lista** de tamanho variável. Há várias formas de montar isso, e a escolha importa:

Opção	Problema
Column	não rola; com conteúdo maior que a tela, dá <i>overflow</i>
SingleChildScrollView + Column	rola, mas constrói todos os itens de uma vez, inclusive os que estão fora da tela
ListView com tudo dentro	rola e é preguiçoso, mas a barra do topo e os blocos fixos viram "itens" misturados com a lista
CustomScrollView + slivers	a escolhida: uma única rolagem, partes com comportamentos diferentes, lista construída sob demanda

Sliver é um "pedaço" de uma área rolável. Cada sliver sabe se comportar durante a rolagem:

```
CustomScrollView
├── SliverAppBar           ← barra que fica fixa (pinned) no topo
├── SliverPadding
│   ├── SliverList.list   ← blocos fixos: saudação, busca, perfil, categorias
│   └── SliverPadding
│       └── SliverList.separated ← a lista: construída sob demanda
```

Regra de ouro dos slivers: dentro de um **CustomScrollView**, os filhos **diretos** precisam ser slivers. Para colocar um widget comum (um **Container**, um **Text**), envolva-o em um **SliverToBoxAdapter** ou use uma lista como **SliverList.list**.

4.2 Passo 1: O esqueleto

Vamos montar a **HomePage** aos poucos. Primeiro, a estrutura com a barra superior e a área de conteúdo. As seções entram nos passos seguintes.

lib/features/home/presentation/pages/home_page.dart (versão inicial)

```
import 'package:flutter/material.dart';

import '../../../app/theme/app_spacing.dart';
import '../../../establishments/domain/establishment_repository.dart';
```

```

class HomePage extends StatefulWidget {
  const HomePage({super.key, required this.repository});

  final EstablishmentRepository repository;

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  @override
  Widget build(BuildContext context) {
    return CustomScrollView(
      slivers: [
        SliverAppBar(
          pinned: true,
          title: Row(
            children: [
              Image.asset(
                'assets/images/brand/logo_celilac.png',
                height: 32,
                excludeFromSemantics: true,
              ),
              const SizedBox(width: AppSpacing.sm),
              const Text(
                'CeliLac',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
            ],
          ),
        ),
        SliverPadding(
          padding: const EdgeInsets.fromLTRB(
            AppSpacing.lg,
            AppSpacing.sm,
            AppSpacing.lg,
            0,
          ),
          sliver: SliverList.list(
            children: const [
              // seções dos passos 2 a 5
            ],
          ),
        ),
      ],
    );
  }
}

```

Conceitos

StatefulWidget desde já. A Home vai guardar o carregamento da lista (Parte 5). Como no shell, o estado fica no **State**, e a configuração imutável (o repositório) fica no widget, acessada como **widget.repository**.

O repositório chega pelo construtor. A Home **recebe** o repositório em vez de criá-lo. É a mesma injeção pelo construtor do **OnboardingStorage**, e é ela que permite, nos testes, entregar à Home um repositório controlado (Parte 7).

SliverAppBar(pinned: true). A barra fica **sempre visível** no topo enquanto o conteúdo rola por baixo. Outras opções: **floating: true** (some ao rolar para baixo, reaparece ao rolar para cima) e nenhuma das duas (rola junto com o conteúdo). Para uma barra só com a marca, **pinned** mantém a identidade sempre presente.

Logo na barra. O mesmo **logo_celilac.png** da Splash, agora pequeno: a identidade continua presente. **excludeFromSemantics: true** porque o texto "CeliLac" ao lado já diz o nome: o leitor de tela não precisa anunciar duas vezes.

Sem Scaffold na Home? O **Scaffold** é do **AppShell**. A Home é o **conteúdo** de uma aba. Ela usa o **Material** e o **ScaffoldMessenger** que estão acima dela na árvore.

Execute: a aba Início mostra a barra com logo e nome, sobre o fundo cinza-claro do tema.

4.3 Passo 2: Saudação

Conceito: lógica pura fora do widget

A saudação muda conforme a hora ("Bom dia", "Boa tarde", "Boa noite"). Essa regra **não depende de Flutter**: recebe uma hora e devolve um texto. Mantê-la em uma **função pura** separada do widget permite **testá-la** sem abrir nenhuma tela (Parte 7).

lib/features/home/presentation/greeting.dart

```
String greetingFor(DateTime time) {
  final hour = time.hour;
  if (hour >= 5 && hour < 12) {
    return 'Bom dia';
  }
  if (hour >= 12 && hour < 18) {
    return 'Boa tarde';
  }
  return 'Boa noite';
}
```

Função pura: o resultado depende **só** dos parâmetros. Por isso ela recebe o **DateTime** em vez de chamar **DateTime.now()** lá dentro. Se chamasse, o teste dependeria do relógio da máquina, e "às 11h59 é bom dia?" seria impossível de verificar.

lib/features/home/presentation/widgets/home_header.dart

```
import 'package:flutter/material.dart';

import '../../../app/common/widgets/gold_accent.dart';
import '../../../app/theme/app_colors.dart';
import '../../../app/theme/app_spacing.dart';

class HomeHeader extends StatelessWidget {
  const HomeHeader({super.key, required this.greeting});

  final String greeting;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          '$greeting!',
          style: textTheme.headlineSmall?.copyWith(
            color: AppColors.navy,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: AppSpacing.sm),
        const GoldAccent(),
        const SizedBox(height: AppSpacing.sm),
        Text(
          'O que você procura hoje?',
          style: textTheme.bodyLarge?.copyWith(
            color: AppColors.navy.withValues(alpha: 0.8),
          ),
        ),
      ],
    );
  }
}
```

```

    ),
  ],
);
}
}

```

Na `HomePage`, primeiro item da lista (adicione o `import` de `greeting.dart` e de `home_header.dart`):

```
HomeHeader(greeting: greetingFor(DateTime.now())),
```

Remova o `const` de `children`: `const` [. No esqueleto, a lista vazia podia ser `const`. O `HomeHeader` recebe o resultado de `greetingFor(DateTime.now())`, que só é conhecido durante a execução, então a lista deixa de poder ser `const`. Deixe `children`: [e marque como `const` só os itens que podem ser (`const SizedBox(...)`, `const SectionHeader(...)`).

Conceitos

- **Mesma linguagem do onboarding**: título `headlineSmall` navy em negrito + `GoldAccent` + texto de apoio. O usuário reconhece o produto.
- **Alinhado à esquerda**, diferente do onboarding (centralizado). O onboarding tinha **uma** mensagem por tela; a Home é uma tela de **leitura e varredura**, e o olho lê melhor quando todos os blocos começam na mesma margem.
- **O `HomeHeader` recebe o texto pronto**. Ele não sabe que horas são; só exhibe. É o "a página decide, o widget renderiza" do material anterior.
- **Por que sem o nome do usuário?** Ainda não há perfil. Uma saudação genérica é melhor do que um "Olá, usuário!" artificial. Quando o perfil existir, `HomeHeader` ganha um parâmetro `name`.

4.4 Passo 3: Entrada da busca

Conceito: parece campo, age como botão

Muitos apps (de entrega, de mapas, de hospedagem) mostram na Home algo que **parece** um campo de busca, mas que, ao ser tocado, **abre uma tela de busca** dedicada. Motivos:

- a busca de verdade precisa de espaço: sugestões, histórico, filtros, resultados;
- se o campo fosse real, o teclado abriria **sobre** a Home e esconderia metade dela;
- o formato de campo é **reconhecido instantaneamente**: ninguém precisa ler para entender.

`lib/features/home/presentation/widgets/search_entry.dart`

```

import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_radius.dart';
import '../theme/app_spacing.dart';

class SearchEntry extends StatelessWidget {
  const SearchEntry({super.key, required this.onTap});

  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      child: Material(
        color: Colors.white,
        borderRadius: BorderRadius.circular(AppRadius.md),
        child: InkWell(
          onTap: onTap,
          borderRadius: BorderRadius.circular(AppRadius.md),
          child: Container(
            constraints: const BoxConstraints(minHeight: 56),

```

```

padding: const EdgeInsets.symmetric(
  horizontal: AppSpacing.md,
  vertical: AppSpacing.sm,
),
child: Row(
  children: [
    const Icon(Icons.search, color: AppColors.navy),
    const SizedBox(width: AppSpacing.sm),
    Expanded(
      child: Text(
        'Buscar produtos ou lugares',
        style: Theme.of(context).textTheme.bodyLarge?.copyWith(
          color: AppColors.navy.withValues(alpha: 0.8),
        ),
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
      ),
    ),
  ],
),
),
),
),
),
),
);
}
}

```

Na **HomePage**, depois do cabeçalho:

```

const SizedBox(height: AppSpacing.lg),
SearchEntry(onTap: () => _showComingSoon('A busca')),

```

O método `_showComingSoon` é criado no Passo 6 (Parte 4.7). Se quiser rodar antes, use `onTap: () {}` temporariamente.

Conceitos

Material + InkWell: o efeito de toque. O **InkWell** desenha a "onda" (*ripple*) do Material ao toque, mas ela é pintada no **Material** mais próximo **acima** dele. Se o fundo branco fosse um **Container** com **BoxDecoration**, a onda ficaria **escondida atrás** da cor. Por isso a cor e o raio vão no **Material**, e o **InkWell** repete o mesmo **borderRadius** para a onda respeitar os cantos.

Semantics(button: true). Para o leitor de tela, isto é um **botão** chamado "Buscar produtos ou lugares". Sem essa marcação, ele leria só o texto, e o usuário não saberia que pode tocar.

constraints: BoxConstraints(minHeight: 56) em vez de height: 56. Altura **mínima**, e não fixa. Com a fonte ampliada nas configurações do aparelho, o texto cresce e a caixa cresce junto. Com **height: 56**, o texto seria cortado em cima e embaixo (Parte 6.2).

VoidCallback onTap. O widget não decide o que acontece ao toque: recebe uma função. Hoje ela mostra "em breve"; amanhã, abre a tela de busca. **O widget não muda.**

Texto do placeholder com navy 80%. O texto de dica tem a mesma regra de contraste de qualquer texto (Parte 1.5). Dicas quase invisíveis (cinza-claro sobre branco) são um erro comum.

4.5 Passo 4: Cartão de perfil

Conceito: um destaque, uma ação

Este cartão leva o usuário a cumprir a promessa 3 do onboarding (personalização). Ele é o **único elemento escuro** da tela: por contraste, é o primeiro a chamar a atenção depois do título. Por isso tem **uma única ação**. Um destaque com dois botões divide a atenção que ele mesmo criou.

Atenção: este cartão fica na aba Início, e não na aba Perfil. Ele é um bloco da **HomePage**, entre a busca e as categorias. A aba **Perfil** da barra inferior continua mostrando a tela "em breve" (Parte 3.3), porque o perfil ainda não foi construído. O cartão é um **convite** para ir ao perfil, e por isso fica onde o usuário passa todos os dias: a Home. Convidar alguém a completar o perfil dentro do próprio perfil não levaria ninguém até lá. Quando a tela de perfil existir, o botão "Completar perfil" vai levar até ela; por enquanto, ele mostra a mensagem "O perfil alimentar estará disponível em breve."

lib/features/home/presentation/widgets/profile_prompt_card.dart

```
import 'package:flutter/material.dart';

import '../../../../../app/theme/app_colors.dart';
import '../../../../../app/theme/app_radius.dart';
import '../../../../../app/theme/app_spacing.dart';

class ProfilePromptCard extends StatelessWidget {
  const ProfilePromptCard({super.key, required this.onPressed});

  final VoidCallback onPressed;

  @override
  Widget build(BuildContext context) {
    final textTheme = Theme.of(context).textTheme;

    return Container(
      padding: const EdgeInsets.all(AppSpacing.lg),
      decoration: BoxDecoration(
        color: AppColors.navy,
        borderRadius: BorderRadius.circular(AppRadius.md),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              const Icon(Icons.tune, color: AppColors.gold),
              const SizedBox(width: AppSpacing.sm),
              Expanded(
                child: Text(
                  'Complete seu perfil alimentar',
                  style: textTheme.titleMedium?.copyWith(
                    color: Colors.white,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
            ],
          ),
          const SizedBox(height: AppSpacing.sm),
          Text(
            'Conte o que você precisa evitar e veja '
            'primeiro as opções mais adequadas.',
            style: textTheme.bodyMedium?.copyWith(
              color: Colors.white.withValues(alpha: 0.85),
              height: 1.4,
            ),
          ),
          const SizedBox(height: AppSpacing.md),
          FilledButton(
            onPressed: onPressed,
            style: FilledButton.styleFrom(
              backgroundColor: AppColors.gold,
              foregroundColor: AppColors.navy,
              minimumSize: const Size(0, 48),
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(AppRadius.sm),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

```

        ),
      ),
      child: const Text(
        'Completar perfil',
        style: TextStyle(fontWeight: FontWeight.bold),
      ),
    ),
  ],
),
);
}
}

```

Na [HomePage](#), depois da busca:

```

const SizedBox(height: AppSpacing.lg),
ProfilePromptCard(
  onPressed: () => _showComingSoon('0 perfil alimentar'),
),

```

Conceitos

O que é *microcopy*. É o nome dado aos **textos curtos da interface**: rótulos de botões, dicas dentro de campos, títulos de cartões, mensagens de erro, de lista vazia e de confirmação. O termo vem do inglês (*micro*, pequeno; *copy*, o texto escrito para um produto ou anúncio) e costuma ser usado sem tradução. Este material já está cheio de exemplos:

Onde	<i>Microcopy</i>
dica da busca	"Buscar produtos ou lugares"
botão do cartão de perfil	"Completar perfil"
erro ao carregar a lista	"Não foi possível carregar os estabelecimentos." + "Tentar novamente"
lista vazia	"Ainda não encontramos opções perto de você."
aviso de recurso futuro	"A busca estará disponível em breve."

São poucas palavras, mas é por elas que o usuário entende **o que pode fazer** e **o que aconteceu**. Um botão "OK" ou uma mensagem "Erro 500" são *microcopy* ruins: não dizem nada. Por isso esses textos merecem o mesmo cuidado que as cores e os espaçamentos.

Microcopy orientada a benefício. No cartão de perfil, o título diz **o que fazer** ("Complete seu perfil alimentar"), e a descrição diz **o que o usuário ganha** ("veja primeiro as opções mais adequadas"). "Configure suas preferências" diria o que fazer, mas não por quê. Sem um motivo, completar o perfil parece só trabalho; com o motivo, parece uma vantagem.

O botão dourado é uma exceção local ao tema. No tema, botões preenchidos são navy (cor primária). **Sobre um fundo navy**, um botão navy desapareceria. Este é o caso legítimo de estilo local da Parte 1.1. As cores seguem a tabela de contraste: texto navy sobre dourado (5,0:1).

minimumSize: Size(0, 48). Altura mínima de 48 dp, a área de toque mínima recomendada pelo Material e pelas diretrizes de acessibilidade. **0** na largura deixa o botão com a largura do texto, alinhado à esquerda como o resto do cartão.

Ícone tune em dourado. O ícone de "ajustes" reforça a ideia de personalizar. Aqui o dourado é **decoração** ao lado de um título branco: o significado está no texto.

Texto branco 85% e height: 1.4. Hierarquia dentro do cartão (título 100%, descrição 85%) com contraste folgado (8,7:1). Entrelinha maior para o texto de duas linhas, como no onboarding.

Quando o perfil existir, a Home deve **esconder** este cartão para quem já completou o perfil. Um convite que não some depois de aceito vira ruído. Veja os desafios na Parte 9.

4.6 Passo 5: Títulos de seção e atalhos de categoria

SectionHeader

lib/features/home/presentation/widgets/section_header.dart

```
import 'package:flutter/material.dart';

import '../../../app/theme/app_colors.dart';

class SectionHeader extends StatelessWidget {
  const SectionHeader({
    super.key,
    required this.title,
    this.actionLabel,
    this.onAction,
  });

  final String title;
  final String? actionLabel;
  final VoidCallback? onAction;

  @override
  Widget build(BuildContext context) {
    final label = actionLabel;

    return Row(
      children: [
        Expanded(
          child: Semantics(
            header: true,
            child: Text(
              title,
              style: Theme.of(context).textTheme.titleMedium?.copyWith(
                color: AppColors.navy,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ),
        if (label != null)
          TextButton(
            onPressed: onAction,
            child: Text(
              label,
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
          ),
      ],
    );
  }
}
```

Conceitos

- **Um widget para todos os títulos de seção.** "Categorias" e "Perto de você" ficam **iguais por construção**. Uma terceira seção no futuro recebe o mesmo visual de graça.
- **Ação opcional.** "Categorias" não tem "Ver todos" (as quatro já estão visíveis); "Perto de você" tem. Parâmetros anuláveis (*String?*) e o **if** dentro da lista de filhos (*collection if*) resolvem os dois casos com um widget só.
- **final label = actionLabel;**. Copiar o campo para uma variável local permite ao Dart **promover** o tipo: depois de **if (label != null)**, **label** é *String*, e não *String?*. Com o campo direto, a promoção não acontece.
- **Semantics(header: true)**. Leitores de tela permitem **pular de título em título**. Marcar os títulos de seção transforma a Home em um documento navegável para quem não enxerga a tela.
- **TextButton herda a cor do tema** (primária = navy). Nenhum estilo de cor local foi necessário.

Ícone por categoria (camada de apresentação)

lib/features/establishments/presentation/establishment_category_icon.dart

```
import 'package:flutter/material.dart';

import '../domain/establishment_category.dart';

extension EstablishmentCategoryIcon on EstablishmentCategory {
  IconData get icon => switch (this) {
    EstablishmentCategory.restaurant => Icons.restaurant_outlined,
    EstablishmentCategory.bakery => Icons.bakery_dining_outlined,
    EstablishmentCategory.cafe => Icons.local_cafe_outlined,
    EstablishmentCategory.market => Icons.storefront_outlined,
  };
}
```

Conceitos

O problema. Cada categoria precisa de um ícone, e o jeito mais natural de escrever isso seria `category.icon`, como já fazemos com `category.label`. Mas não podemos colocar o ícone **dentro** do enum: `IconData` vem do Flutter, e o domínio não importa Flutter (Parte 2.2). Ou seja, queremos um membro novo em um tipo que **não devemos alterar**.

Extensão (*extension method*). É o recurso do Dart (desde a versão 2.7) que resolve exatamente isso: ele **acrescenta métodos e getters a um tipo que já existe, sem modificar o arquivo desse tipo**. O tipo original continua igual; quem **importa** a extensão passa a enxergar os membros novos, como se eles sempre tivessem estado lá.

Anatomia:

Trecho	Significado
<code>extension</code>	palavra que inicia a declaração
<code>EstablishmentCategoryIcon</code>	nome da extensão. É opcional, mas convém dar um: ele aparece nas mensagens de erro e permite resolver conflitos
<code>on EstablishmentCategory</code>	o tipo que ganha os membros novos
<code>IconData get icon => ...</code>	o membro acrescentado: aqui, um <i>getter</i>
<code>this</code>	dentro da extensão, é o objeto em que o membro foi chamado. Em <code>EstablishmentCategory.cafe.icon</code> , <code>this</code> é <code>EstablishmentCategory.cafe</code>

O resultado: `category.icon` funciona em qualquer arquivo que importe `establishment_category_icon.dart`, e o arquivo `establishment_category.dart` continua sem nenhum *import* do Flutter. **Cada camada fica com o que é seu:** o domínio diz quais categorias existem e como se chamam; a apresentação diz como elas aparecem.

Regras que costumam surpreender:

- **A extensão só existe onde é importada.** Esquecer o *import* gera *"The getter 'icon' isn't defined for the type 'EstablishmentCategory'"*, mesmo com a extensão escrita corretamente em outro arquivo. Repare que a própria mensagem sugere *"Try importing the library that defines 'icon'"*.
- **Extensões não têm campos.** Elas podem acrescentar métodos, *getters*, *setters* e operadores, mas não dados guardados no objeto (`int counter = 0;` dá o erro *"Extensions can't declare instance fields"*). Se você precisa guardar algo, o lugar é o próprio tipo.
- **A escolha é feita pelo tipo declarado, durante a compilação.** Em uma variável *dynamic*, o Dart não sabe que tipo ela tem, e a extensão não é encontrada: `category.icon` funciona, mas `(category as dynamic).icon` falha com o app rodando (*NoSuchMethodError*).

Quando usar uma extensão:

Situação	Exemplo
o tipo não é seu e você não pode alterá-lo (vem do Dart, do Flutter ou de um pacote)	<code>extension on String { String capitalize() ... }</code> , ou <code>context.textTheme</code> em vez de <code>Theme.of(context).textTheme</code>
o tipo é seu, mas o membro pertence a outra camada	o nosso caso: o ícone é apresentação, e o enum é domínio
uma conversão usada só em uma camada	na camada <code>data</code> , um <code>toJson()</code> em um modelo do domínio, sem que o domínio saiba que JSON existe

Quando não usar:

- **Quando o membro pertence ao tipo e não há impedimento.** O `label` fica **dentro** do enum, e não em uma extensão: é texto puro do Dart, faz parte do significado da categoria e não fere nenhuma regra de camada. Extensão não substitui escrever o membro no lugar certo.
- **Quando é preciso guardar estado.** Extensões não têm campos.
- **Quando o comportamento muda conforme a subclasse.** Como a extensão é escolhida pelo tipo declarado, ela não tem polimorfismo: em uma variável declarada como `Animal`, é sempre a extensão de `Animal` que roda, mesmo que o objeto seja um `Dog`. Para isso, use um método na classe, sobrescrito na subclasse.
- **Para "enfeitar" tipos muito comuns sem necessidade.** Dezenas de extensões em `String` ou `int` poluem o autocompletar do editor em todo o projeto e escondem a lógica longe de onde se esperaria encontrá-la.

Regra prática: antes de criar uma extensão, pergunte *"posso e devo colocar este membro no próprio tipo?"*. Se a resposta for sim, coloque-o lá. A extensão é para quando a resposta for não: o tipo não é seu, ou o membro é de outra camada.

switch como expressão (Dart 3) e exaustividade. Cada caso devolve um valor, sem `break` e sem `return`. E o mais importante: o compilador **verifica que todos os valores do enum foram tratados**. Se alguém criar `EstablishmentCategory.iceCream` e esquecer o ícone, **o app não compila**. Com um `Map<EstablishmentCategory, IconData>`, o esquecimento só apareceria em tempo de execução.

CategoryShortcuts

`lib/features/home/presentation/widgets/category_shortcuts.dart`

```
import 'package:flutter/material.dart';

import '../../../../../app/theme/app_colors.dart';
import '../../../../../app/theme/app_radius.dart';
import '../../../../../app/theme/app_spacing.dart';
import '../../../../../establishments/domain/establishment_category.dart';
import '../../../../../establishments/presentation/establishment_category_icon.dart';

class CategoryShortcuts extends StatelessWidget {
  const CategoryShortcuts({super.key, required this.onSelected});

  final ValueChanged<EstablishmentCategory> onSelected;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        for (final category in EstablishmentCategory.values)
          Expanded(
            child: _CategoryTile(
              category: category,
              onTap: () => onSelected(category),
            ),
          ),
      ],
    );
  }
}
```

```

class _CategoryTile extends StatelessWidget {
  const _CategoryTile({required this.category, required this.onTap});

  final EstablishmentCategory category;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(AppRadius.sm),
      child: Padding(
        padding: const EdgeInsets.symmetric(vertical: AppSpacing.sm),
        child: Column(
          children: [
            Container(
              width: 56,
              height: 56,
              decoration: const BoxDecoration(
                color: Colors.white,
                shape: BoxShape.circle,
              ),
              child: Icon(category.icon, color: AppColors.navy),
            ),
            const SizedBox(height: AppSpacing.sm),
            Text(
              category.label,
              style: Theme.of(context).textTheme.labelMedium
                ?.copyWith(color: AppColors.navy),
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
            ),
          ],
        ),
      ),
    );
  }
}

```

Na [HomePage](#), depois do cartão de perfil:

```

const SizedBox(height: AppSpacing.xl),
const SectionHeader(title: 'Categorias'),
const SizedBox(height: AppSpacing.sm),
CategoryShortcuts(
  onSelected: (category) =>
    _showComingSoon('A categoria ${category.label}'),
),

```

Conceitos

ValueChanged<EstablishmentCategory>. É um apelido para `void Function(EstablishmentCategory)`. Diferente do `VoidCallback`, ele **informa qual** categoria foi tocada. O componente avisa "tocaram em Padaria", e a página decide o que fazer.

for dentro da lista (collection for). Gera um `Expanded` para cada valor do enum, sem `map(...).toList()`.

Row + Expanded: quatro colunas iguais. Cada atalho ocupa **um quarto** da largura disponível, em qualquer tamanho de tela. Como o conjunto é **pequeno e fixo** (4 itens), não há rolagem horizontal.

Se as categorias passarem de 5, os atalhos ficariam estreitos demais. Aí o certo é uma lista horizontal: `SizedBox(height: 104, child: ListView.separated(scrollDirection: Axis.horizontal, ...))`. Uma `ListView` horizontal **dentro** de uma rolagem vertical precisa de **altura definida** (o `SizedBox`); sem ela, o Flutter não sabe que altura dar à lista e lança um erro de layout.

CategoryTile privado. Só faz sentido dentro de **CategoryShortcuts**, então fica no mesmo arquivo, com **** (visível só nele). O material anterior fez o mesmo com **_OnboardingContent**.

Ícone em círculo branco sobre fundo cinza. O mesmo recurso dos cartões: o branco separa do fundo sem precisar de sombra. O círculo de 56 dp, junto com o rótulo e o **padding**, garante uma área de toque bem acima de 48 dp.

maxLines: 1 + ellipsis. Com fonte muito ampliada, "Restaurante" pode não caber em um quarto da tela. Cortar com reticências é melhor do que quebrar o layout (Parte 6.2).

4.7 Passo 6: Feedback para o que ainda não existe

_HomePageState:

```
void _showComingSoon(String feature) {
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(
      SnackBar(content: Text('$feature estará disponível em breve.')),
    );
}
```

Conceitos

SnackBar. Uma mensagem curta e temporária na parte de baixo da tela, que **não bloqueia** o uso do app. É o componente certo para avisos informativos. Um diálogo (**AlertDialog**) exigiria um toque para fechar, um custo alto para uma informação simples.

ScaffoldMessenger. Quem exibe *snackbars* é o **ScaffoldMessenger** criado pelo **MaterialApp**, e não a tela. Por isso a Home (que não tem **Scaffold** próprio) consegue exibi-los.

hideCurrentSnackBar() antes de mostrar. *Snackbars* entram em uma **fila**. Sem esta linha, cinco toques rápidos geram cinco mensagens, uma depois da outra, por vários segundos. Com ela, a mensagem nova **substitui** a anterior.

Operador cascata (..). Chama vários métodos no **mesmo objeto**: **ScaffoldMessenger.of(context)** é obtido uma vez, e **hideCurrentSnackBar** e **showSnackBar** são chamados nele em sequência.

Mensagem com sujeito. "A busca estará disponível em breve." diz **o que** não existe ainda. "Em breve!" sozinho deixaria a dúvida: o que é que vem em breve?

Execute: toque na busca, no botão do perfil e em cada categoria. Cada toque mostra uma mensagem flutuante, que substitui a anterior.

Parte 5: Dados assíncronos: "Perto de você"

5.1 Conceito: os quatro estados de uma lista

Quando os dados vêm de fora (rede, disco), a tela **não** tem um estado só. Ela tem pelo menos quatro, e **cada um precisa de uma interface**:

Estado	Quando	O que mostrar	Erro comum
Carregando	a busca começou e ainda não terminou	um "esqueleto" do conteúdo	tela vazia (parece quebrado)
Erro	a busca falhou	o que houve + como resolver ("Tentar novamente")	<i>crash</i> , ou mensagem técnica ("SocketException")
Vazio	a busca funcionou, mas não há itens	uma mensagem que explique a ausência	lista vazia sem nada (parece erro)
Dados	a busca funcionou e há itens	a lista	é o único estado que costuma ser feito

Iniciantes costumam programar só o **último** estado, porque é o único que aparece no computador do desenvolvedor, com internet rápida e dados de exemplo. Em uso real, os outros três **acontecem todo dia**: metrô sem sinal, cidade sem estabelecimentos cadastrados, servidor fora do ar.

5.2 Formatação: números para pessoas

lib/app/common/formatters.dart

```
String formatDecimal(double value, {int digits = 1}) {
  return value.toStringAsFixed(digits).replaceAll('.', ',');
}

String formatDistance(double kilometers) {
  if (kilometers < 1) {
    return '${(kilometers * 1000).round()}\u00A0m';
  }
  return '${formatDecimal(kilometers)}\u00A0km';
}
```

Conceitos

- **Vírgula decimal.** Em português, 4,8 e não 4.8. `toStringAsFixed(1)` fixa uma casa decimal, e `replaceAll` troca o separador.
- **Metros abaixo de 1 km.** "0,4 km" obriga a fazer conta; "350 m" é imediato. A unidade muda conforme a grandeza, como fazem os apps de mapa.
- **\u00A0: espaço não separável.** Entre o número e a unidade usamos um espaço que **impede a quebra de linha**. Sem ele, em uma linha apertada, apareceria "1,2" no fim de uma linha e "km" sozinho no começo da seguinte (aconteceu durante a construção deste material).
- **Por que em app/common?** Formatar números não é assunto de estabelecimentos. Qualquer feature futura (preços, avaliações) pode reutilizar.

Dica: para apps com vários idiomas, o pacote `intl` (`NumberFormat`) formata números conforme a região automaticamente. Para um app só em português, as duas funções acima bastam e não acrescentam dependências.

5.3 Selo de opção alimentar: DietaryBadge

lib/features/establishments/presentation/widgets/dietary_badge.dart

```
import 'package:flutter/material.dart';

import '../..../app/theme/app_colors.dart';
import '../..../app/theme/app_radius.dart';
import '../..../app/theme/app_spacing.dart';
import '../..../domain/dietary_option.dart';

class DietaryBadge extends StatelessWidget {
  const DietaryBadge({super.key, required this.option});

  final DietaryOption option;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(
        horizontal: AppSpacing.sm,
        vertical: AppSpacing.xs,
      ),
      decoration: BoxDecoration(
        color: AppColors.gold.withValues(alpha: 0.18),
        borderRadius: BorderRadius.circular(AppRadius.pill),
      ),
      child: Row(
```

```

        mainAxisSize: MainAxisSize.min,
        children: [
          const Icon(Icons.check_circle, size: 14, color: AppColors.navy),
          const SizedBox(width: AppSpacing.xs),
          Flexible(
            child: Text(
              option.label,
              style: Theme.of(context).textTheme.labelSmall?.copyWith(
                color: AppColors.navy,
                fontWeight: FontWeight.w600,
              ),
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
            ),
          ),
        ],
      ),
    ],
  );
}
}

```

Conceitos

Esta é a informação mais importante do cartão. Para quem tem doença celíaca ou intolerância à lactose, "Sem glúten" não é um detalhe: é o **critério de escolha**. Por isso ele tem forma própria (pílula), ícone de confirmação e fundo de destaque.

Dourado como fundo, navy como texto. Dourado a 18% sobre branco vira um bege claro; o texto navy sobre ele tem contraste de 9,9:1. É a regra 2 da Parte 1.5 aplicada.

Ícone de check + texto. A informação não depende só da cor (quem não distingue cores) nem só do ícone (quem não conhece o símbolo).

Linguagem positiva. "✓ Sem glúten" (o que **é seguro**), e não "× Glúten" (o que é proibido). O material anterior evitou símbolos de proibição nas ilustrações pelo mesmo motivo: o app comunica **possibilidades**, não restrições.

mainAxisSize: MainAxisSize.min + Flexible. A pílula tem a largura do conteúdo (**min**). O **Flexible** permite que o texto **encolha** com reticências se não houver espaço, em vez de transbordar. Esse ajuste foi descoberto **pelos testes automatizados** (Parte 7): a fonte de teste é mais larga e revelou o **overflow** que aconteceria com fonte ampliada.

5.4 O cartão: EstablishmentCard

lib/features/establishments/presentation/widgets/establishment_card.dart

```

import 'package:flutter/material.dart';

import '../common/formatters.dart';
import '../theme/app_colors.dart';
import '../theme/app_radius.dart';
import '../theme/app_spacing.dart';
import '../domain/establishment.dart';
import '../establishment_category_icon.dart';
import 'dietary_badge.dart';

class EstablishmentCard extends StatelessWidget {
  const EstablishmentCard({
    super.key,
    required this.establishment,
    required this.onTap,
  });

  final Establishment establishment;
  final VoidCallback onTap;

  @override

```

```

Widget build(BuildContext context) {
  final textTheme = Theme.of(context).textTheme;
  final details = [
    establishment.category.label,
    establishment.neighborhood,
    formatDistance(establishment.distanceInKm),
  ].join(' · ');

  return MergeSemantics(
    child: Card(
      clipBehavior: Clip.antiAlias,
      child: InkWell(
        onTap: onTap,
        child: Padding(
          padding: const EdgeInsets.all(AppSpacing.md),
          child: Row(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Container(
                width: 56,
                height: 56,
                decoration: BoxDecoration(
                  color: AppColors.gold.withValues(alpha: 0.18),
                  borderRadius: BorderRadius.circular(AppRadius.sm),
                ),
                child: Icon(
                  establishment.category.icon,
                  color: AppColors.navy,
                ),
              ),
              const SizedBox(width: AppSpacing.md),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      establishment.name,
                      style: textTheme.titleMedium?.copyWith(
                        color: AppColors.navy,
                        fontWeight: FontWeight.bold,
                      ),
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                    ),
                    const SizedBox(height: AppSpacing.xs),
                    Text(
                      details,
                      style: textTheme.bodySmall?.copyWith(
                        color: AppColors.navy.withValues(alpha: 0.8),
                      ),
                    ),
                    const SizedBox(height: AppSpacing.sm),
                    Wrap(
                      spacing: AppSpacing.xs,
                      runSpacing: AppSpacing.xs,
                      children: [
                        for (final option in establishment.dietaryOptions)
                          DietaryBadge(option: option),
                      ],
                    ),
                  ],
                ),
              ),
              const SizedBox(width: AppSpacing.sm),
              Row(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [

```

```

const Icon(
  Icons.star_rounded,
  size: 18,
  color: AppColors.gold,
  semanticLabel: 'Nota',
),
const SizedBox(width: 2),
Text(
  formatDecimal(establishment.rating),
  style: textTheme.labelLarge?.copyWith(
    color: AppColors.navy,
    fontWeight: FontWeight.bold,
  ),
),
),
),
),
),
),
),
),
),
),
),
),
);
}
}

```

Conceitos

Anatomia e hierarquia.

	Café Aconchego	★ 4,8	← 1. nome (o que é)	4. nota
	Café · Centro · 350 m		← 2. contexto (onde, quão longe)	
	[✓ Sem glúten] [✓ Sem lactose]		← 3. segurança (posso ir?)	

O nome é o maior e mais forte; o contexto é menor e mais claro; os selos têm forma própria. Em uma lista, o olho **varre** os nomes e só lê o resto do que interessa.

Card + InkWell + clipBehavior: Clip.antiAlias. O **Card** é um **Material**, então a onda do **InkWell** aparece sobre o branco dele (como explicado na Parte 4.4). O **clipBehavior** recorta a onda nos **cantos arredondados**; sem ele, ela "vaza" em forma retangular.

Sem cor, sem forma, sem margem no Card. Tudo vem do **cardTheme** (Parte 1.3). Se um dia os cartões mudarem de raio, muda em um lugar.

join(' · '). Monta "Café · Centro · 350 m" a partir de uma lista. O ponto médio (·) é um separador leve, comum em interfaces, que não compete com o conteúdo.

Expanded na coluna central. O ícone à esquerda e a nota à direita têm largura fixa; o texto ocupa **o que sobra**. Sem o **Expanded**, um nome longo empurraria a nota para fora da tela.

Wrap para os selos. Se os dois selos não couberem lado a lado, o **Wrap** leva o segundo para a **linha de baixo**. Uma **Row** daria *overflow*.

maxLines: 2 no nome. Nomes longos ganham uma segunda linha antes de serem cortados.

MergeSemantics: um cartão, uma leitura. Sem ele, o leitor de tela pararia em cada pedaço: "Café Aconchego" → "Café · Centro · 350 m" → "Sem glúten" → ... Com ele, o cartão vira **um único item**, lido de uma vez e acionado com um toque duplo.

Por que o MergeSemantics fica por fora do Card e do InkWell? Ele junta tudo o que está **abaixo** dele. Se ficasse dentro do **InkWell**, juntaria só os textos, e o **InkWell** (que é quem responde ao toque) ficaria de fora, como um item separado e **sem nome**: o leitor de tela anunciaria um botão mudo e, depois, um texto que não se pode tocar. Por fora, os textos e a ação de toque viram o

mesmo item. Esse erro foi encontrado por uma verificação automática de acessibilidade (Parte 6.3) em uma versão anterior deste material.

semanticLabel: 'Nota'. A estrela é só um desenho; para o leitor de tela, ela passa a dizer "Nota", e o resultado é "Nota, 4,8" em vez de um "4,8" sem contexto.

5.5 Estados de carregamento, erro e vazio

lib/features/home/presentation/widgets/nearby_states.dart

```
import 'package:flutter/material.dart';

import '../../../../../app/theme/app_colors.dart';
import '../../../../../app/theme/app_radius.dart';
import '../../../../../app/theme/app_spacing.dart';

class NearbyLoading extends StatelessWidget {
  const NearbyLoading({super.key});

  @override
  Widget build(BuildContext context) {
    return Semantics(
      label: 'Carregando estabelecimentos',
      child: Column(
        children: [
          for (var i = 0; i < 3; i++) ...[
            if (i > 0) const SizedBox(height: AppSpacing.md),
            Container(
              height: 96,
              decoration: BoxDecoration(
                color: AppColors.navy.withValues(alpha: 0.06),
                borderRadius: BorderRadius.circular(AppRadius.md),
              ),
            ),
          ],
        ],
      ),
    );
  }
}

class NearbyMessage extends StatelessWidget {
  const NearbyMessage({
    super.key,
    required this.icon,
    required this.message,
    this.actionLabel,
    this.onAction,
  });

  final IconData icon;
  final String message;
  final String? actionLabel;
  final VoidCallback? onAction;

  @override
  Widget build(BuildContext context) {
    final label = actionLabel;

    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(AppSpacing.lg),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(AppRadius.md),
      ),
    );
  }
}
```

```

    ),
    child: Column(
      children: [
        Icon(icon, size: 40, color: AppColors.navy),
        const SizedBox(height: AppSpacing.sm),
        Text(
          message,
          style: Theme.of(context).textTheme.bodyMedium
            ?.copyWith(color: AppColors.navy.withValues(alpha: 0.8)),
          textAlign: TextAlign.center,
        ),
        if (label != null) ...[
          const SizedBox(height: AppSpacing.sm),
          TextButton(onPressed: onAction, child: Text(label)),
        ],
      ],
    ),
  );
}
}

```

Atenção: este arquivo fica em `home/presentation/widgets/`, e não em `establishments/`. Os textos ("Perto de você", "Tentar novamente") são da seção da **Home**, e não do assunto "estabelecimento" (regra da Parte 0.5). O teste da Parte 7.4 importa o arquivo deste caminho.

Conceitos

Esqueleto (*skeleton*) em vez de *spinner*. Três blocos com o **formato aproximado** dos cartões. Comparado a um círculo girando:

- o usuário já vê **como** será o conteúdo, e a espera parece menor;
- quando os dados chegam, os cartões ocupam o lugar dos blocos, **sem "pulo"** de layout.

Semantics (label: ...). Blocos cinza não dizem nada a quem usa leitor de tela. O rótulo anuncia "Carregando estabelecimentos".

...[] (spread) dentro do for. Cada volta do **for** gera **dois** widgets (o espaço e o bloco). O *spread* "despeja" a lista interna na lista de filhos. O **if (i > 0)** evita um espaço antes do primeiro bloco.

Um widget para erro e vazio. Os dois estados têm a mesma estrutura (ícone + mensagem + ação opcional). O que muda é o conteúdo, e quem decide o conteúdo é a página.

Mensagens que ajudam.

Estado	Mensagem	Por quê
Erro	"Não foi possível carregar os estabelecimentos." + Tentar novamente	diz o que falhou sem culpar o usuário e oferece a saída
Vazio	"Ainda não encontramos opções perto de você."	"ainda" indica que é uma situação temporária, não um defeito

Nunca mostre `snapshot.error.toString()` ao usuário. "Exception: SocketException: Failed host lookup" não ajuda ninguém e pode expor detalhes internos. Em produção, registre o erro técnico em uma ferramenta de monitoramento e mostre uma mensagem humana.

5.6 FutureBuilder: como funciona e o erro mais comum

Relembrando: o que é um **Future**

Um **Future<T>** é a **promessa de um valor que ainda não existe**. Quando a Home chama `widget.repository.fetchNearby()`, a lista de estabelecimentos não volta na hora: volta um **Future<List<Establishment>>**, que é como uma senha de restaurante. Com ela na mão, você sabe que o pedido **vai ficar pronto**, mas ainda não tem a comida. Mais tarde, o **Future completa** de um de dois jeitos:

- **com um valor:** a lista chegou;
- **com um erro:** a busca falhou (sem internet, servidor fora do ar).

Em código comum, esperamos um **Future** com **await**. Mas o método **build** **não pode esperar**: ele precisa devolver um widget imediatamente, porque o Flutter desenha a tela dezenas de vezes por segundo. A pergunta é: *o que mostrar enquanto o **Future** não completa, e como trocar a tela quando ele completar?*

O problema sem o **FutureBuilder**

Sem ajuda, teríamos de controlar tudo à mão no **State**:

```
// Funciona, mas é trabalhoso e fácil de errar
bool _isLoading = true;
Object? _error;
List<Establishment>? _data;

@override
void initState() {
  super.initState();
  widget.repository.fetchNearby().then(
    (data) => setState(() { _data = data; _isLoading = false; }),
    onError: (Object error) => setState(() { _error = error; _isLoading = false; }),
  );
}
```

Três campos para manter coerentes, um **setState** em cada caminho, e ainda faltaria tratar o caso de a tela ser fechada antes de o **Future** completar (um **setState** depois disso gera erro). Esse código se repetiria em toda tela que busca dados.

O que o **FutureBuilder** faz

O **FutureBuilder** é um widget do Flutter que **faz esse trabalho por você**: ele recebe um **Future**, fica "escutando" esse **Future** e **reconstrói a parte da tela que você indicar** cada vez que a situação muda. Ele recebe dois parâmetros principais:

Parâmetro	O que é
future	o Future a acompanhar
builder	uma função (context , snapshot) => Widget , chamada para desenhar a tela em cada momento do Future

O **snapshot** (do inglês, "instantâneo", como uma foto) é um **AsyncSnapshot<T>**: uma **fotografia da situação do **Future** naquele instante**. O **builder** olha a foto e decide o que desenhar:

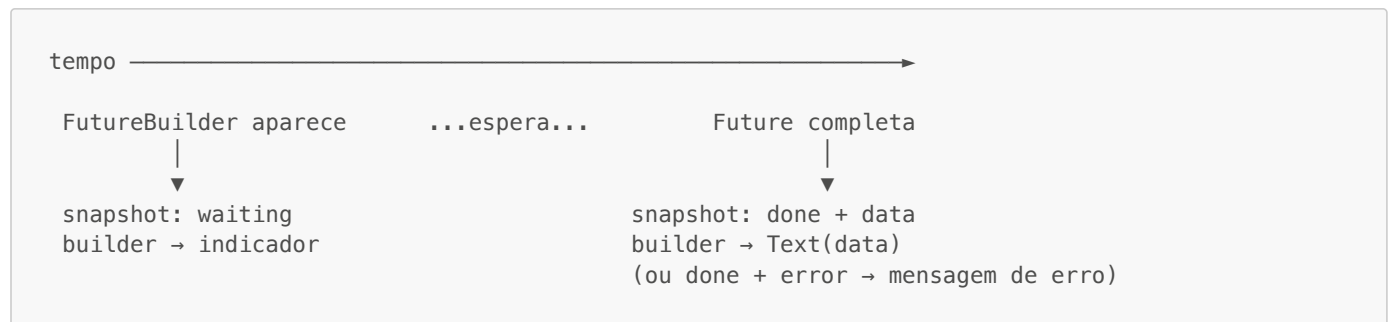
Propriedade	O que informa
connectionState	em que fase está: none (não há Future), waiting (em andamento) ou done (completou). Existe também active , usado só com Stream , no StreamBuilder
hasData / data	se há um valor, e qual é
hasError / error	se completou com erro, e qual foi

Um exemplo mínimo, fora dos slivers:

```
FutureBuilder<String>(
  future: _messageFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState != ConnectionState.done) {
      return const CircularProgressIndicator();
    }
    if (snapshot.hasError) {
      return const Text('Não foi possível carregar.');
```

```
    },  
  )
```

Ao longo do tempo, o **builder** é chamado com fotos diferentes:



O **FutureBuilder** cuida sozinho do **setState**, dos três campos e do caso de a tela ser fechada antes da hora. A tela só descreve **o que mostrar em cada situação**.

Como ele reconhece "o mesmo **Future**"

Este é o detalhe que explica o erro mais comum. O **FutureBuilder** é um widget, e widgets são **recriados a cada build**. A cada nova construção, ele compara o **Future** que recebeu com o da construção anterior **pela identidade**: é o **mesmo objeto** na memória?

- **Mesmo objeto**: nada muda; ele continua esperando ou mostrando o resultado que já tem.
- **Outro objeto**: ele **abandona** o **Future** anterior (ignora o resultado dele, mesmo que chegue depois), volta para **waiting** e passa a acompanhar o novo.

Dois **Futures** criados por duas chamadas a **fetchNearby()** são **objetos diferentes**, mesmo que vão devolver a mesma lista.

O erro mais comum

Sabendo disso, a forma "óbvia" de usar um **FutureBuilder** se revela **errada**:

```
// Errado: NÃO faça isso
FutureBuilder(
  future: widget.repository.fetchNearby(), // dentro do build!
  builder: ...
)
```

O **build** roda **muitas vezes**: ao mostrar um **snackbar**, ao girar a tela, ao abrir o teclado, quando um pai se reconstrói. Cada execução chama **fetchNearby()** de novo, e cada chamada cria um **Future novo**, ou seja, uma **nova requisição**. O **FutureBuilder** vê um objeto diferente, abandona o anterior e recomeça a espera. O servidor recebe dezenas de pedidos repetidos, e, dependendo de como o **builder** foi escrito, a lista pisca em "carregando" sem motivo. Os testes da Parte 7 **provam** isso: com esse código, uma única reconstrução fez o repositório ser chamado 3 vezes.

O certo é **criar o Future uma vez**, no **initState**, guardá-lo no estado e entregar ao **FutureBuilder** **sempre o mesmo objeto**.

Adicione ao _HomePageState (no início da classe) o campo e o **initState** abaixo, com o **import** de **establishment.dart**. Este passo é obrigatório: **_buildNearby**, **_reload** e **_refresh** (Partes 5.7 e 5.8) usam **_nearbyFuture**, e sem esta declaração o **flutter analyze** acusa **"Undefined name '_nearbyFuture'"**.

```
class _HomePageState extends State<HomePage> {
  late Future<List<Establishment>> _nearbyFuture;

  @override
  void initState() {
    super.initState();
    _nearbyFuture = widget.repository.fetchNearby();
  }
}
```

```
// ...
}
```

late. O campo não tem valor na declaração, mas receberá um **antes do primeiro uso** (no `initState`, que roda antes do `build`). Não é **late final** porque o `Future` será **substituído** ao recarregar (Parte 5.8).

5.7 FutureBuilder dentro dos slivers

Adicione ao `_HomePageState` (com os *imports* de `establishment_card.dart` e de `../widgets/nearby_states.dart`):

```
Widget _buildNearby() {
  return FutureBuilder<List<Establishment>>(
    future: _nearbyFuture,
    builder: (context, snapshot) {
      final isLoading =
        snapshot.connectionState != ConnectionState.done &&
        !snapshot.hasData;

      if (isLoading) {
        return const SliverToBoxAdapter(child: NearbyLoading());
      }

      if (snapshot.hasError) {
        return SliverToBoxAdapter(
          child: NearbyMessage(
            icon: Icons.cloud_off_outlined,
            message: 'Não foi possível carregar os estabelecimentos.',
            actionLabel: 'Tentar novamente',
            onAction: _reload,
          ),
        );
      }

      final establishments = snapshot.data ?? const <Establishment>[];

      if (establishments.isEmpty) {
        return const SliverToBoxAdapter(
          child: NearbyMessage(
            icon: Icons.search_off_outlined,
            message: 'Ainda não encontramos opções perto de você.',
          ),
        );
      }

      return SliverList.separated(
        itemCount: establishments.length,
        itemBuilder: (context, index) {
          final establishment = establishments[index];
          return EstablishmentCard(
            establishment: establishment,
            onTap: () => _showComingSoon('0 detalhe do estabelecimento'),
          );
        },
        separatorBuilder: (_, _) => const SizedBox(height: AppSpacing.md),
      );
    },
  );
}
```

Conceitos

Os quatro estados no `builder`. O funcionamento do `FutureBuilder` e do `snapshot` foi explicado na Parte 5.6. Aqui, o `builder` traduz a foto do `snapshot` nos quatro estados da Parte 5.1, **nesta ordem**: carregando → erro → vazio → dados. Cada `if` termina com `return`, e o código lê como uma tabela de decisão.

Por que `isLoading` olha também `!hasData`? Ao recarregar (Parte 5.8), o `FutureBuilder` recebe um `Future` novo e volta para `waiting`, mas **mantém os dados (e o erro) anteriores no `snapshot`**. Se olhássemos só o `connectionState`, a lista sumiria e daria lugar ao esqueleto a cada atualização, um "pisca" desnecessário. Com a condição dupla:

- primeira carga (sem dados): mostra o esqueleto;
- recarga (com dados): **mantém a lista** visível enquanto o indicador de atualização gira.

Por que o teste de carregamento vem antes do teste de erro? Pelo mesmo motivo: o `snapshot` guarda o erro anterior. Ao tocar em "Tentar novamente" depois de uma falha, o `snapshot` fica em `waiting`, **ainda com o erro antigo**. Se o `if` (`snapshot.hasError`) viesse primeiro, a mensagem de erro continuaria na tela durante a nova tentativa. Testando o carregamento antes, o usuário vê o esqueleto, que é o que de fato está acontecendo.

O `builder` devolve slivers. O `FutureBuilder` está no lugar de um sliver (dentro do `SliverPadding`), então **todos** os retornos precisam ser slivers: `SliverToBoxAdapter` para os estados de um widget só, `SliverList.separated` para a lista.

`SliverList.separated`: construção sob demanda. O `itemBuilder` só é chamado para os cartões **perto da área visível**. Com 4 itens, a diferença é pequena; com 400, é a diferença entre um app fluido e um travado. O `separatorBuilder` coloca o espaço **entre** os itens (e não depois do último).

`snapshot.data ?? const <Establishment>[]`. Garante uma lista não nula. O tipo explícito (`<Establishment>`) é necessário porque uma lista vazia sozinha não diz de que tipo é.

5.8 Recarregar: "Tentar novamente" e puxar para atualizar

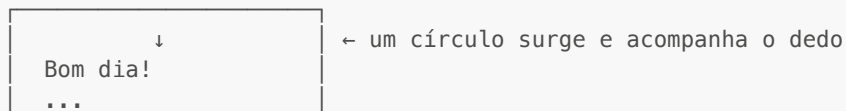
Dados que vêm de fora envelhecem: um estabelecimento novo foi cadastrado, a busca falhou porque o metrô estava sem sinal. O usuário precisa de um jeito de dizer "busque de novo". A Home oferece **dois**:

Forma	Quando aparece	Como o usuário aciona
botão "Tentar novamente"	só no estado de erro	um toque no botão
puxar para atualizar	sempre , com ou sem dados	arrastar a tela para baixo estando no topo

O que é "puxar para atualizar"

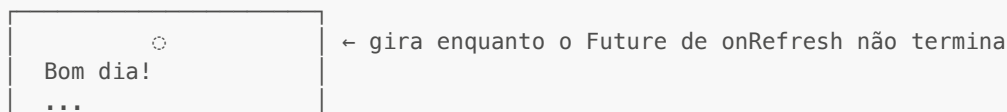
É o gesto, presente em quase todo app de celular (e-mail, redes sociais, bancos), de **arrastar o conteúdo para baixo quando ele já está no topo** para buscar dados novos. Em inglês, *pull to refresh*. Passo a passo, no Flutter:

1. O usuário está no topo da lista e arrasta para baixo.



2. Se arrastar além de uma distância mínima, o indicador fica "armado". Se soltar antes disso, ele volta e nada acontece.

3. Ao soltar, o indicador começa a girar e o Flutter chama `onRefresh`.



4. Quando o `Future` termina (com sucesso ou com erro), o indicador some.

No Flutter, esse comportamento vem pronto no widget `RefreshIndicator`, que envolve a área rolável (veja a Parte 5.9). Ele precisa de uma única coisa nossa: a função `onRefresh`, que **inicia a atualização e devolve um `Future` que só termina quando a atualização terminar**. É por esse `Future` que ele sabe **até quando** girar.

Adicione ao `_HomePageState`:

```
void _reload() {
  setState(() {
    _nearbyFuture = widget.repository.fetchNearby();
  });
}

Future<void> _refresh() async {
  _reload();
  try {
    await _nearbyFuture;
  } catch (_) {
    // O erro já é exibido pelo FutureBuilder.
  }
}
```

Conceitos

Recarregar = trocar o Future. O `FutureBuilder` percebe que recebeu outro objeto e passa a acompanhá-lo. O `setState` é o que faz o `build` rodar com o `Future` novo.

_reload para o botão "Tentar novamente". Um toque, uma nova tentativa. O `FutureBuilder` volta ao esqueleto (não há dados) e depois mostra o resultado. O botão não precisa saber quando a busca termina, por isso `_reload` não devolve nada (`void`).

_refresh para o gesto de puxar. O `RefreshIndicator` precisa de uma função que devolva um `Future` e mantém o indicador girando **até ele terminar**. `_refresh` faz duas coisas: chama `_reload` (que troca o `Future` e inicia a busca) e depois **espera** esse mesmo `Future` com `await`. Quando a busca termina, `_refresh` termina, e o indicador some.

Por que dois métodos, e não um? Os dois recarregam do mesmo jeito; a diferença é **quem chama e o que essa pessoa precisa saber**. O botão só precisa disparar a busca; o `RefreshIndicator` precisa saber quando ela acaba. Por isso `_refresh` reaproveita `_reload` e só acrescenta a espera: a lógica de recarregar fica em um único lugar.

Não há busca duplicada. `_refresh` espera o mesmo objeto `_nearbyFuture` que acabou de entregar ao `FutureBuilder`. Os dois acompanham a mesma requisição: um para desenhar o resultado, o outro para saber quando parar de girar. Se `_refresh` chamasse `fetchNearby()` de novo em vez de esperar `_nearbyFuture`, seriam **duas** requisições.

O que o usuário vê durante a atualização. Como a lista já tem dados, ela **continua visível** enquanto o indicador gira (é o efeito da condição `!snapshot.hasData` explicada na Parte 5.7). Quando os dados novos chegam, os cartões são atualizados no lugar, sem passar pelo esqueleto.

Por que o try/catch vazio? Se o carregamento falhar, o `await` **relança** o erro dentro de `_refresh`. Sem o `catch`, esse erro chegaria ao `RefreshIndicator`, que não sabe tratá-lo, e apareceria como erro não tratado no console. O erro **já** está sendo exibido ao usuário pelo `FutureBuilder`; aqui, só precisamos que o indicador pare de girar. O comentário explica a intenção: um `catch` vazio **sem** comentário parece descuido.

O gesto é invisível, então não pode ser o único caminho. Nada na tela avisa que dá para puxar: quem não conhece o padrão nunca vai descobri-lo, e para quem usa leitor de tela ou tem dificuldade motora o gesto pode ser difícil de fazer. Por isso, no momento em que recarregar é **necessário** (o erro), a Home mostra um botão visível, "Tentar novamente". Puxar para atualizar é um **atalho** para quem já conhece o gesto, e não a única saída.

No iPhone, o `RefreshIndicator` mostra o círculo do Material, diferente do indicador nativo do iOS. Se quiser o visual de cada plataforma, troque por `RefreshIndicator.adaptive(...)`, que usa o indicador do iOS no iPhone e o do Material no Android.

Avisos esperados neste ponto. `_buildNearby` e `_refresh` já existem, mas o `build` ainda não os usa. Por isso o `flutter analyze` mostra *"The declaration '_buildNearby' isn't referenced"* e o mesmo para `_refresh`. São **avisos**, não erros, e somem no próximo passo, quando o `build` passa a chamar os dois.

5.9 O build completo da HomePage

Esta é a versão final do arquivo. O `build` substitui o da Parte 4: ele envolve tudo em um `RefreshIndicator`, acrescenta o título "Perto de você" e um segundo `SliverPadding` com `_buildNearby()`. Confira também os *imports* (entra `dart:math`) e o campo `maxContentWidth` no widget.

`lib/features/home/presentation/pages/home_page.dart` (versão final)

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../../../../../app/theme/app_spacing.dart';
import '../../../../../establishments/domain/establishment.dart';
import '../../../../../establishments/domain/establishment_repository.dart';
import '../../../../../establishments/presentation/widgets/establishment_card.dart';
import '../greeting.dart';
import '../widgets/category_shortcuts.dart';
import '../widgets/home_header.dart';
import '../widgets/nearby_states.dart';
import '../widgets/profile_prompt_card.dart';
import '../widgets/search_entry.dart';
import '../widgets/section_header.dart';

class HomePage extends StatefulWidget {
  const HomePage({super.key, required this.repository});

  final EstablishmentRepository repository;

  static const double maxContentWidth = 600;

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  late Future<List<Establishment>> _nearbyFuture;

  @override
  void initState() {
    super.initState();
    _nearbyFuture = widget.repository.fetchNearby();
  }

  void _reload() {
    setState(() {
      _nearbyFuture = widget.repository.fetchNearby();
    });
  }

  Future<void> _refresh() async {
    _reload();
    try {
      await _nearbyFuture;
    } catch (_) {
      // O erro já é exibido pelo FutureBuilder.
    }
  }

  void _showComingSoon(String feature) {
    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(content: Text('$feature estará disponível em breve.')),
      );
  }

  @override
```

```

Widget build(BuildContext context) {
  final screenWidth = MediaQuery.sizeOf(context).width;
  final horizontalPadding = math.max(
    AppSpacing.lg,
    (screenWidth - HomePage.maxContentWidth) / 2,
  );

  return RefreshIndicator(
    onRefresh: _refresh,
    child: CustomScrollView(
      physics: const AlwaysScrollableScrollPhysics(),
      slivers: [
        SliverAppBar(
          pinned: true,
          title: Row(
            children: [
              Image.asset(
                'assets/images/brand/logo_celilac.png',
                height: 32,
                excludeFromSemantics: true,
              ),
              const SizedBox(width: AppSpacing.sm),
              const Text(
                'CeliLac',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
            ],
          ),
        ),
        SliverPadding(
          padding: EdgeInsets.fromLTRB(
            horizontalPadding,
            AppSpacing.sm,
            horizontalPadding,
            0,
          ),
          sliver: SliverList.list(
            children: [
              HomeHeader(greeting: greetingFor(DateTime.now())),
              const SizedBox(height: AppSpacing.lg),
              SearchEntry(onTap: () => _showComingSoon('A busca')),
              const SizedBox(height: AppSpacing.lg),
              ProfilePromptCard(
                onPressed: () => _showComingSoon('O perfil alimentar'),
              ),
              const SizedBox(height: AppSpacing.xl),
              const SectionHeader(title: 'Categorias'),
              const SizedBox(height: AppSpacing.sm),
              CategoryShortcuts(
                onSelected: (category) =>
                  _showComingSoon('A categoria ${category.label}'),
              ),
              const SizedBox(height: AppSpacing.lg),
              SectionHeader(
                title: 'Perto de você',
                actionLabel: 'Ver todos',
                onAction: () => _showComingSoon('A lista completa'),
              ),
              const SizedBox(height: AppSpacing.sm),
            ],
          ),
        ),
        SliverPadding(
          padding: EdgeInsets.fromLTRB(
            horizontalPadding,
            0,
            horizontalPadding,

```

```

        AppSpacing.xl,
      ),
      sliver: _buildNearby(),
    ),
  ],
),
);
}

// _buildNearby(): código da Parte 5.7
}

```

A estrutura

RefreshIndicator	← puxar para atualizar
└ CustomScrollView (sempre rolável)	
└ SliverAppBar (pinned)	← logo + nome, fixa
└ SliverPadding	
└ SliverList.list	← blocos fixos
└ HomeHeader	← saudação
└ SearchEntry	← busca
└ ProfilePromptCard	← próximo passo
└ SectionHeader	← "Categorias"
└ CategoryShortcuts	← 4 atalhos
└ SectionHeader	← "Perto de você" + "Ver todos"
└ SliverPadding	
└ FutureBuilder	← 4 estados
└ SliverList.separated	← cartões sob demanda

Conceitos

RefreshIndicator por fora de tudo. Ele "escuta" a rolagem do filho: quando o usuário puxa o topo para baixo, mostra o indicador e chama `onRefresh`. É o gesto que usuários de celular esperam para "atualizar".

AlwaysScrollableScrollPhysics. Por padrão, uma área rolável cujo conteúdo **cabe** na tela não rola, e então não dá para puxar. Em um tablet, ou com a lista vazia, o conteúdo pode caber. Esta física garante que o gesto **sempre** funcione.

O ritmo dos espaços. `lg` (24) entre blocos relacionados ao topo; `xl` (32) antes de uma **nova seção**; `sm` (8) entre o título da seção e o seu conteúdo. Como na Parte 5.6 do material anterior: **espaços maiores separam, espaços menores agrupam**.

Dois SliverPadding. O primeiro envolve os blocos fixos, e o segundo, a lista. O espaço de `xl` no fim do segundo afasta o último cartão da barra de navegação.

horizontalPadding é explicado na Parte 6.1.

Execute e teste:

- ao abrir, três blocos cinza aparecem por menos de um segundo e dão lugar aos cartões;
- role: a barra superior fica fixa e ganha uma sombra suave;
- puxe a lista para baixo a partir do topo: o indicador gira e a lista se mantém visível;
- simule erro no *fake* (Parte 2.5): aparece a mensagem com "Tentar novamente";
- simule lista vazia: aparece a mensagem de vazio;
- **desfaça** as simulações.

Parte 6: Qualidade: responsividade e acessibilidade

Até aqui, a Home foi construída e conferida em **uma** situação: um celular comum, em pé, com a fonte padrão, usado por alguém que enxerga a tela e toca nela com precisão. Esta parte trata de todas as outras situações. São dois assuntos diferentes, mas com a mesma pergunta de fundo: **a tela continua funcionando quando as condições mudam?**

	Responsividade	Acessibilidade
o que muda	o espaço disponível (tamanho da tela, orientação, janela)	a pessoa que usa e o jeito como ela usa (visão, fonte, coordenação motora, leitor de tela)
pergunta	o layout se adapta a telas pequenas, grandes, em pé e deitadas?	qualquer pessoa consegue ler, entender e acionar tudo?
seções	6.1	6.2, 6.3, 6.4 (e o contraste, na Parte 1.5)

6.1 Responsividade: largura máxima de leitura

O que é responsividade

Uma interface **responsiva** é a que **se ajusta ao espaço disponível**, em vez de ser desenhada para um único tamanho. O mesmo app do CeliLac pode rodar em:

- um celular pequeno (320 dp de largura) ou grande (430 dp);
- um celular deitado (a largura passa a ser a altura);
- um tablet (800 dp ou mais);
- uma janela redimensionável (tela dividida no Android, desktop, navegador).

Lembre que **dp** não é pixel (Parte 1.2): 360 dp é a largura de um celular comum, qualquer que seja a resolução dele.

Há dois níveis de adaptação, e a documentação do Flutter usa os dois termos:

	Responsivo	Adaptativo
o que faz	o mesmo layout se estica, encolhe ou reflui	um layout diferente para cada faixa de tamanho
exemplo	os atalhos de categoria dividem a largura em quatro, seja ela qual for	em tablets, trocar a NavigationBar de baixo por uma NavigationRail na lateral

As diretrizes do Material 3 dividem as larguras em **classes de tamanho de janela**: *compact* (menos de 600 dp, a maioria dos celulares em pé), *medium* (600 a 839 dp, tablets em pé e celulares dobráveis abertos) e *expanded* (840 dp ou mais, tablets deitados e desktop). Esses limites são chamados de *breakpoints* (pontos de quebra): onde vale a pena mudar o layout.

A Home do CeliLac é **responsiva**: um único layout, que funciona em qualquer largura. Layouts adaptativos (como a **NavigationRail** em tablets) ficam como próximo passo (Parte 8). O ajuste que fazemos aqui é o mais importante para telas largas: limitar a largura do conteúdo.

O problema das linhas longas

Em um tablet (ou celular deitado), os cartões esticados de ponta a ponta ficam **difíceis de ler**: o olho percorre uma linha longa demais e se perde na volta. Limitamos o conteúdo a **600 dp** (o limite da classe *compact*) e centralizamos:

```
static const double maxContentWidth = 600;

// no build:
final screenWidth = MediaQuery.sizeOf(context).width;
final horizontalPadding = math.max(
  AppSpacing.lg,
  (screenWidth - HomePage.maxContentWidth) / 2,
);
```

Largura da tela	Cálculo	Margem lateral
360 (celular)	$(360 - 600) / 2 = -120 \rightarrow \text{máx}(24, -120)$	24
800 (tablet)	$(800 - 600) / 2 = 100 \rightarrow \text{máx}(24, 100)$	100 (conteúdo com 600)

Conceitos

- **Margem calculada em vez de `ConstrainedBox`.** Dentro de um `CustomScrollView`, não dá para envolver slivers em um `ConstrainedBox` (ele é um widget comum, não um sliver). Calcular a margem do `SliverPadding` alcança o mesmo resultado, e a **área de rolagem continua ocupando a tela toda**: o usuário rola mesmo tocando nas laterais.
- **`MediaQuery.sizeOf(context)` em vez de `MediaQuery.of(context).size`.** O primeiro reconstrói a Home **só** quando o tamanho muda; o segundo, a cada mudança de **qualquer** informação do `MediaQuery` (teclado, brilho, escala de texto...).
- **`import 'dart:math' as math;`** O prefixo `math.` deixa claro de onde vem `max`, e evita conflito com outros nomes.

Execute: confira a Home em pelo menos três tamanhos:

- **celular deitado:** gire o emulador (botões de rotação na barra lateral do emulador Android, ou `Cmd + →` no simulador do iOS);
- **tablet:** crie um emulador de tablet no Android Studio (*Device Manager* → *Create Virtual Device* → *Tablet*) e rode o app nele;
- **janela redimensionável:** rode com `flutter run -d macos` (ou `-d windows`, `-d chrome`) e arraste a borda da janela. A margem lateral deve crescer quando a janela passa de 600 dp, e o conteúdo deve ficar centralizado.

O que é acessibilidade

Acessibilidade é a qualidade de um app que **pode ser usado por todas as pessoas**, inclusive pessoas com deficiência. A Organização Mundial da Saúde estima que mais de um bilhão de pessoas no mundo têm alguma deficiência, e no Brasil a Lei Brasileira de Inclusão (Lei 13.146/2015) trata do direito de acesso à informação e à comunicação.

Mas acessibilidade não é só para "outras pessoas". As limitações podem ser:

Tipo	Visão	Movimento
permanente	cegueira, baixa visão, daltonismo	tremor, amputação
temporária	olho dilatado depois de um exame	braço engessado
situacional	sol forte na tela	uma mão segurando a sacola, a outra o celular

Quem tem doença celíaca ou intolerância à lactose (o público do CeliLac) inclui pessoas de todas as idades, e a baixa visão é comum entre pessoas mais velhas. Um app acessível é, simplesmente, um app que funciona para mais gente.

Neste material, a acessibilidade aparece em quatro frentes:

Frente	Para quem	Onde
contraste de cores	baixa visão, daltonismo, sol forte	Parte 1.5
fonte ampliada	baixa visão, pessoas mais velhas	Parte 6.2
leitor de tela	pessoas cegas ou com baixa visão severa	Parte 6.3
áreas de toque	tremor, coordenação motora, dedos grandes, pressa	Parte 6.4

6.2 Fonte ampliada

Muitas pessoas usam a fonte do celular **aumentada** (a configuração existe exatamente para isso). O Flutter respeita essa configuração automaticamente: todo `Text` é multiplicado pelo fator escolhido pelo usuário. O app precisa **continuar funcionando**: nenhum texto pode ser cortado de forma que impeça o uso, e nada pode transbordar.

Como ativar, para testar:

- **Android:** *Configurações* → *Acessibilidade* → *Tamanho da fonte* (o caminho varia um pouco conforme o fabricante; em alguns aparelhos fica em *Configurações* → *Tela*). Arraste para o maior valor.
- **iOS:** *Ajustes* → *Acessibilidade* → *Tela e Tamanho do Texto* → *Texto Maior*. Ative "Tamanhos Maiores" para chegar aos valores extremos.

Testamos a Home com fonte a **200%** em telas de **320 dp** (o pior caso comum) e de **800 dp** de largura, rolando a tela inteira. Resultado: **nenhum overflow**. As decisões que garantiram isso:

Decisão	Onde	Efeito
<code>minHeight</code> em vez de <code>height</code>	<code>SearchEntry</code>	a caixa cresce com o texto
<code>Expanded</code> no texto central	cartão, busca, título de seção	o texto usa o espaço que sobra, sem empurrar vizinhos

Decisão	Onde	Efeito
<code>Wrap</code> nos selos	cartão	selos descem de linha
<code>Flexible + ellipsis</code>	selo	o texto encolhe com reticências
<code>maxLines: 2</code>	nome no cartão	uma linha extra antes de cortar
<code>CustomScrollView</code>	página toda	a tela rola, e conteúdo maior nunca transborda

Não trave a escala da fonte (`MediaQuery(textScaler: TextScaler.noScaling)`) para "proteger o layout". Para quem precisa da fonte grande, isso torna o app **inutilizável**. O caminho é o layout se adaptar.

6.3 Leitor de tela

O que é um leitor de tela

Um **leitor de tela** é um programa do sistema operacional que **lê em voz alta** o que está na tela e permite **operar o aparelho sem enxergá-lo**. É assim que pessoas cegas usam celulares todos os dias: para mandar mensagens, pagar contas e, no nosso caso, escolher onde comer.

O uso muda a forma de tocar na tela:

Gesto	Sem leitor de tela	Com leitor de tela
um toque	aciona o elemento	seleciona o elemento e lê o que ele é ("Completar perfil, botão")
toque duplo (em qualquer lugar)	—	aciona o elemento selecionado
deslizar um dedo para a direita / esquerda	rola ou troca de página	vai para o próximo / anterior elemento, em ordem de leitura
deslizar dois dedos	—	rola a tela

Os dois leitores de tela mais usados vêm instalados nos próprios sistemas:

- **TalkBack** (Android): *Configurações* → *Acessibilidade* → *TalkBack*. Dica: na mesma tela, ative o **atalho** (segurar as duas teclas de volume por alguns segundos), para ligar e desligar o TalkBack rapidamente durante os testes. Em emuladores **sem** Google Play, o TalkBack pode não vir instalado; prefira um aparelho físico ou uma imagem de emulador com Google Play.
- **VoiceOver** (iOS): *Ajustes* → *Acessibilidade* → *VoiceOver*. Também dá para configurar o atalho em *Ajustes* → *Acessibilidade* → *Atalho de Acessibilidade* (clique triplo no botão lateral). O **simulador do iOS não tem VoiceOver**; use um iPhone físico.

Antes de testar, treine. Com o leitor de tela ligado, o celular se comporta de outro jeito, e é fácil ficar "preso". Pratique primeiro na tela inicial do aparelho: deslize para a direita para ir de item em item, toque duas vezes para abrir, e use o atalho para desligar.

Como o Flutter "conversa" com o leitor de tela: a árvore de semântica

Um app Android ou iOS comum é feito de componentes do próprio sistema, e o leitor de tela já sabe o que cada um é. O Flutter é diferente: ele **desenha** a interface inteira por conta própria, pixel a pixel. Para o sistema operacional, a tela de um app Flutter é, à primeira vista, só uma imagem.

Por isso, ao lado da árvore de widgets, o Flutter monta uma segunda árvore, a **árvore de semântica** (*semantics tree*), e a entrega ao sistema. Cada nó dessa árvore descreve um pedaço da tela **com palavras**: o que é (botão, título, imagem), qual o seu texto, e o que se pode fazer com ele (tocar, rolar). É essa árvore que o TalkBack e o VoiceOver leem.

Os widgets do Material já preenchem a árvore sozinhos: um `FilledButton` se anuncia como botão com o seu texto, um `Text` vira um nó com aquele texto. O nosso trabalho é **corrigir e completar** a árvore onde o automático não basta:

Recurso	Onde	Problema que resolve	Resultado
<code>excludeFromSemantics: true</code>	logo da barra	a imagem e o texto "CeliLac" diriam o nome duas vezes	o nome é lido uma vez

Recurso	Onde	Problema que resolve	Resultado
<code>Semantics(header: true)</code>	títulos de seção	um título seria lido como texto comum	o usuário pode pular de título em título
<code>Semantics(button: true)</code>	busca	parece campo, mas é tocável; seria lido como texto	anunciada como botão
<code>MergeSemantics</code>	cartão	o cartão seria lido em 5 ou 6 pedaços soltos	o cartão é um item , lido de uma vez e acionado com toque duplo
<code>semanticLabel: 'Nota'</code>	estrela	a estrela é só desenho; "4,8" ficaria sem contexto	"Nota, 4,8"
<code>Semantics(label: ...)</code>	esqueleto	blocos cinza não dizem nada	"Carregando estabelecimentos"

Ferramentas para verificar

O teste definitivo é usar o app com o leitor de tela ligado, mas ele é lento. Estas ferramentas ajudam no dia a dia:

1. O depurador de semântica do Flutter. Ligue no `MaterialApp`, temporariamente:

```
MaterialApp(
  showSemanticsDebugger: true, // só durante o desenvolvimento
  // ...
)
```

A tela passa a mostrar, no lugar do app, **caixas com o texto de cada nó da árvore de semântica**: exatamente o que o leitor de tela vai encontrar. Um elemento tocável sem texto, ou um cartão quebrado em vários pedaços, fica visível na hora. **Remova a linha** depois de verificar.

2. Verificações automáticas do `flutter_test`. O pacote de testes do Flutter já traz regras de acessibilidade prontas, que podem ser verificadas em um teste de widget (Parte 7):

```
testWidgets('a Home segue as regras de acessibilidade', (tester) async {
  final handle = tester.ensureSemantics();
  await _pumpHome(tester, repo);
  await tester.pumpAndSettle();

  await expectLater(tester, meetsGuideline(androidTapTargetGuideline));
  await expectLater(tester, meetsGuideline(iOSTapTargetGuideline));
  await expectLater(tester, meetsGuideline(labeledTapTargetGuideline));
  await expectLater(tester, meetsGuideline(textContrastGuideline));
  handle.dispose();
});
```

Regra	O que verifica
<code>androidTapTargetGuideline</code>	áreas de toque com pelo menos 48 × 48 dp
<code>iOSTapTargetGuideline</code>	áreas de toque com pelo menos 44 × 44 pt
<code>labeledTapTargetGuideline</code>	todo elemento tocável tem um texto para o leitor de tela
<code>textContrastGuideline</code>	contraste mínimo entre texto e fundo

Rodamos essas quatro regras na Home. A de rótulos **encontrou um erro real**: na primeira versão do cartão, o `MergeSemantics` estava dentro do `InkWell`, e a área de toque do cartão ficava sem nome (veja a Parte 5.4). Depois da correção, as quatro passam.

3. Ferramentas dos sistemas. No Android, o app gratuito **Accessibility Scanner** (Play Store) analisa a tela aberta e aponta áreas de toque pequenas, contraste baixo e elementos sem rótulo. No Mac, o **Accessibility Inspector** do Xcode (Xcode → *Open Developer Tool* → *Accessibility Inspector*) mostra o que o VoiceOver enxerga em cada elemento.

Execute: com o TalkBack ou o VoiceOver ligado, percorra a Home deslizando para a direita, do topo até o último cartão. Confira se:

- o logo **não** é anunciado separadamente do nome;
- "Categorias" e "Perto de você" são anunciados como **título**;
- a busca é anunciada como **botão**;
- cada cartão é lido **de uma vez** (nome, detalhes, selos e "Nota, 4,8") e um toque duplo nele mostra a mensagem "em breve".

6.4 Áreas de toque

Todo elemento tocável tem **pelo menos 48 x 48 dp** (recomendação do Material; no iOS, a Apple recomenda 44 x 44 pt). Um alvo pequeno é difícil de acertar para quem tem tremor ou pouca coordenação motora, e irritante para qualquer pessoa com pressa ou usando o celular com uma só mão.

Na Home: a busca tem 56 de altura, os atalhos têm o círculo de 56 mais o rótulo, o botão do perfil tem `minimumSize` de 48, os cartões inteiros são tocáveis e os `TextButton` do Material já respeitam o mínimo. As regras `androidTapTargetGuideline` e `iOSTapTargetGuideline` (Parte 6.3) confirmam isso automaticamente.

6.5 Por que a Home não tem animação de entrada

Splash e onboarding têm animações de entrada elaboradas. A Home, **intencionalmente**, não tem. O material anterior falou em "movimento com moderação"; aqui o critério é a **frequência**:

Tela	Vista	Animação de entrada
Splash	toda abertura, por segundos	sim: é o momento da marca
Onboarding	uma vez na vida	sim: é a primeira impressão
Home	várias vezes por dia	não: na décima vez, é só espera

O movimento da Home é **funcional**: a onda do toque, o indicador de atualização, a troca do esqueleto pelos cartões. Movimento que **responde** ao usuário, e não que **o faz esperar**.

Parte 7: Testes automatizados (conteúdo complementar)

Conteúdo complementar, para investigação. Esta parte **não é necessária** para a Home funcionar: o app das Partes 1 a 6 está completo sem ela. Ela está aqui para quem quiser **aprofundar** e dar o próximo passo rumo a um projeto profissional. Leia, rode os testes, quebre o código de propósito para vê-los falhar, e investigue a documentação indicada no fim desta parte. Se o tempo for curto, pule para a Parte 8 e volte aqui depois.

7.1 Conceito: o que é um teste automatizado, e por que testar

Testar à mão x testar com código

Até aqui, "testar" significou **rodar o app e olhar**: abrir a Home, ver se os cartões aparecem, simular um erro no *fake*, conferir a mensagem. Isso é um **teste manual**, e continua necessário. Mas ele tem limites:

- **depende de memória**: a cada mudança, alguém precisa lembrar de **todos** os casos (e o caso esquecido é justamente o que quebra);
- **demora**: conferir os quatro estados da lista à mão leva minutos, toda vez;
- **não pega regressões**: uma **regressão** é algo que funcionava e **parou de funcionar** depois de uma mudança em outro lugar. Ninguém volta a testar à mão o que "já estava pronto".

Um **teste automatizado** é um **pequeno programa que verifica outro programa**. Ele executa um pedaço do código do app, com uma situação preparada, e **confere se o resultado é o esperado**. Se for, o teste passa; se não for, ele falha e diz o que estava errado. Como é código, o computador o executa em segundos, quantas vezes for preciso, sem esquecer nenhum caso.

A estrutura de um teste: preparar, agir, verificar

Quase todo teste segue os mesmos três passos (em inglês, *Arrange, Act, Assert*):

```
test('formata distâncias menores que 1 km em metros', () {
  // 1. Preparar: a situação de partida
  const distancia = 0.35;

  // 2. Agir: executar o código que está sendo testado
  final texto = formatDistance(distancia);

  // 3. Verificar: comparar com o resultado esperado
  expect(texto, '350 m');
});
```

A frase do **test** descreve a **regra** que está sendo verificada. Se um dia alguém mudar **formatDistance** e quebrar essa regra, o terminal mostra exatamente essa frase, com o valor esperado e o valor obtido. O teste funciona como uma **documentação que se verifica sozinha**.

Por que a Home é um bom ponto de partida

A Home é a primeira tela com **lógica que vale verificar**: regras de horário, formatação de números e quatro estados de carregamento. E o repositório injetado pelo construtor (Parte 4.2) torna a tela **testável**: nos testes, entregamos um repositório que responde exatamente o que queremos (sucesso, lista vazia ou erro), sem depender de internet ou de esperar 800 ms.

Os tipos de teste no Flutter

Tipo	O que testa	Velocidade	Exemplo
Unidade	uma função ou classe, sem interface	milissegundos	greetingFor , formatDistance
Widget	uma tela ou componente, em um ambiente simulado (sem aparelho)	~1 segundo	os 4 estados da HomePage
Integração	o app inteiro, em um aparelho ou emulador	minutos	(fora do escopo deste material)

Um projeto saudável tem **muitos** testes de unidade (rápidos e baratos), **vários** de widget e **poucos** de integração (lentos, mas os mais próximos do uso real). Essa proporção costuma ser desenhada como uma **pirâmide**, com os testes de unidade na base.

7.2 Configuração: **flutter_test**

O **pubspec.yaml** do CeliLac **não tem** o **flutter_test** (projetos criados com **flutter create** normalmente o trazem). Sem ele, **flutter test** falha com *"cannot run without a dependency on flutter_test"*. Adicione:

```
dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
# ...
```

E rode **flutter pub get**.

7.3 Testes de unidade

test/features/home/greeting_test.dart

```
import 'package:celilac/features/home/presentation/greeting.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  DateTime at(int hour) => DateTime(2026, 1, 1, hour);

  test('retorna "Bom dia" das 5h às 11h59', () {
```

```

    expect(greetingFor(at(5)), 'Bom dia');
    expect(greetingFor(at(11)), 'Bom dia');
  });

  test('retorna "Boa tarde" das 12h às 17h59', () {
    expect(greetingFor(at(12)), 'Boa tarde');
    expect(greetingFor(at(17)), 'Boa tarde');
  });

  test('retorna "Boa noite" das 18h às 4h59', () {
    expect(greetingFor(at(18)), 'Boa noite');
    expect(greetingFor(at(0)), 'Boa noite');
    expect(greetingFor(at(4)), 'Boa noite');
  });
}

```

test/app/formatters_test.dart

```

import 'package:celilac/app/common/formatters.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('formata distâncias menores que 1 km em metros', () {
    expect(formatDistance(0.35), '350\u00A0m');
  });

  test('formata distâncias a partir de 1 km com vírgula decimal', () {
    expect(formatDistance(1.2), '1,2\u00A0km');
    expect(formatDistance(1), '1,0\u00A0km');
  });
}

```

Conceitos

- **test(descrição, corpo) e expect(valor, esperado)**. A descrição em português, como uma frase, vira documentação: ao falhar, o terminal mostra **qual regra** quebrou.
- **Testar as fronteiras**. Os erros moram nas bordas: 11h é "bom dia", mas 12h já é "boa tarde"? 4h ainda é "boa noite"? Testar só "10h" não pegaria um `<=` trocado por `<`.
- **at(int hour)**: uma função auxiliar deixa cada linha do teste curta e legível.
- A estrutura de **test/ espelha a de lib/**. Encontrar o teste de um arquivo fica imediato.

7.4 Testes de widget da HomePage

test/features/home/home_page_test.dart

```

import 'package:celilac/app/theme/app_theme.dart';
import 'package:celilac/features/establishments/domain/dietary_option.dart';
import 'package:celilac/features/establishments/domain/establishment.dart';
import 'package:celilac/features/establishments/domain/establishment_category.dart';
import 'package:celilac/features/establishments/domain/establishment_repository.dart';
import 'package:celilac/features/home/presentation/pages/home_page.dart';
import 'package:celilac/features/home/presentation/widgets/nearby_states.dart';
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

class _StubRepository implements EstablishmentRepository {
  _StubRepository(this._results);

  final List<Future<List<Establishment>> Function()> _results;
  int calls = 0;

  @override

```

```

Future<List<Establishment>> fetchNearby() => _results[calls++]()
}

const _cafe = Establishment(
  id: '1',
  name: 'Café Teste',
  category: EstablishmentCategory.cafe,
  neighborhood: 'Centro',
  distanceInKm: 0.35,
  rating: 4.8,
  dietaryOptions: {DietaryOption.glutenFree},
);

Future<void> _pumpHome(WidgetTester tester, EstablishmentRepository repo) {
  tester.view.physicalSize = const Size(1080, 2400);
  tester.view.devicePixelRatio = 3;
  addTearDown(tester.view.reset);

  return tester.pumpWidget(
    MaterialApp(
      theme: AppTheme.light,
      home: Scaffold(body: HomePage(repository: repo)),
    ),
  );
}

void main() {
  testWidgets('mostra carregamento e depois os estabelecimentos', (
    tester,
  ) async {
    final repo = _StubRepository([
      () async => [_cafe],
    ]);
    await _pumpHome(tester, repo);

    expect(find.byType(NearbyLoading), findsOne);

    await tester.pumpAndSettle();
    await tester.ensureVisible(find.text('Café Teste'));
    await tester.pumpAndSettle();

    expect(find.text('Café Teste'), findsOne);
    expect(find.text('Café · Centro · 350\u00A0m'), findsOne);
    expect(find.text('Sem glúten'), findsOne);
  });

  testWidgets('mostra mensagem quando não há estabelecimentos', (tester) async {
    final repo = _StubRepository([() async => []]);
    await _pumpHome(tester, repo);
    await tester.pumpAndSettle();

    final message = find.text('Ainda não encontramos opções perto de você.');
```

```

    await tester.pumpAndSettle();
    await tester.tap(retry);
    await tester.pumpAndSettle();

    expect(repo.calls, 2);
    expect(find.text('Café Teste'), findsOne);
  });

  testWidgets('não busca de novo quando a tela é reconstruída', (tester) async {
    final repo = _StubRepository([
      () async => [_cafe],
    ]);
    await _pumpHome(tester, repo);
    await tester.pumpAndSettle();

    // Bombear a mesma árvore de novo força um novo build() da HomePage.
    await _pumpHome(tester, repo);
    await tester.pumpAndSettle();

    expect(repo.calls, 1);
  });

  testWidgets('avisa que o perfil ainda não está disponível', (tester) async {
    final repo = _StubRepository([
      () async => [_cafe],
    ]);
    await _pumpHome(tester, repo);
    await tester.pumpAndSettle();

    await tester.tap(find.text('Completar perfil'));
    await tester.pump();

    expect(
      find.text('0 perfil alimentar estará disponível em breve.'),
      findsOne,
    );
  });
}

```

Conceitos

O repositório substituto (*stub*). `_StubRepository` implementa o mesmo contrato da Parte 2.4. A `HomePage` não percebe a diferença, e o teste controla tudo: cada chamada a `fetchNearby` devolve o próximo resultado da lista `_results` (sucesso, vazio ou erro), e `calls` conta **quantas vezes** a tela pediu dados. É aqui que a separação em camadas **se paga**.

Funções que criam *Futures*, e não *Futures* prontos. `_results` guarda `() async => ...`. Um *Future* que falha criado **antes** do teste geraria um erro "não tratado" logo na criação. Criando-o só quando a Home pede, o erro acontece no momento certo, e o `FutureBuilder` o trata.

O ambiente simulado.

Comando	O que faz
<code>tester.pumpWidget(...)</code>	constrói a árvore de widgets (um "abrir a tela")
<code>tester.pump()</code>	avança um quadro
<code>tester.pumpAndSettle()</code>	avança quadros até não haver mais nada acontecendo (animações, <i>Futures</i>)
<code>find.text</code> / <code>find.byType</code>	localiza widgets na árvore
<code>tester.ensureVisible</code>	rola até o widget ficar visível
<code>tester.tap</code>	toca no centro do widget

Tamanho de tela realista. O ambiente de teste usa, por padrão, uma "tela" de 800 × 600. `tester.view.physicalSize` com `devicePixelRatio: 3` simula um celular de 360 × 800 dp. O `addTearDown(tester.view.reset)` devolve o tamanho padrão no fim, para não afetar outros testes.

Por que `ensureVisible` antes de tocar? A lista é construída **sob demanda** (Parte 5.7). Um item logo abaixo da tela pode já existir na árvore, mas um toque nele "cairia fora da tela". Rolar até ele imita o que o usuário faria.

O teste mais importante: "não busca de novo". Ele **prova** a regra da Parte 5.6. Chamar `_pumpHome` de novo reconstrói a `HomePage` com a mesma árvore; se o `Future` fosse criado no `build`, o repositório seria chamado de novo. Durante a construção deste material, trocamos de propósito o código pela versão errada, e o teste **falhou** com `Expected: <1>, Actual: <3>`. Um teste que nunca falhou não prova nada; este já provou.

Um assunto por teste. Cada `testWidgets` verifica **um** comportamento e tem um nome que o descreve. Quando um falhar, o nome já diz o que quebrou.

7.5 Rodando os testes

```
flutter test
```

Resultado esperado:

```
+10: All tests passed!
```

E a análise estática:

```
flutter analyze
```

```
No issues found!
```

No seu projeto: comece com **um** teste de unidade (uma função pura sua) e **um** teste de widget (o estado de erro da sua lista). Rode `flutter test` antes de cada entrega.

Para investigar

- **Documentação oficial de testes do Flutter:** docs.flutter.dev/testing/overview (visão geral) e docs.flutter.dev/cookbook/testing (receitas passo a passo de unidade, widget e integração).
- **Testes de integração:** o pacote `integration_test`, que roda o app em um aparelho de verdade.
- **Dublês de teste:** o `stub` desta parte é feito à mão; pacotes como `mocktail` geram substitutos automaticamente. Compare as duas abordagens.
- **Testes de acessibilidade:** as regras `meetsGuideline` da Parte 6.3 podem virar um teste permanente em `home_page_test.dart`.
- **Cobertura:** rode `flutter test --coverage` e descubra quais linhas do app nenhum teste executa.

Parte 8: Revisão: decisões, alternativas e próximos passos

Um trabalho profissional explica não só o que foi feito, mas **o que foi deixado para depois e por quê**.

Decisão	Alternativa	Quando mudar
<code>FutureBuilder</code> + <code>setState</code>	gerência de estado (Provider, Riverpod, Bloc)	quando os mesmos dados forem usados por várias telas (ex.: favoritos na Home e na aba Favoritos)

Decisão	Alternativa	Quando mudar
Repositório criado no <code>AppShell</code>	injeção de dependências (<code>get_it</code> , <code>Provider</code>)	quando houver vários repositórios e serviços
<code>FakeEstablishmentRepository</code>	<code>ApiEstablishmentRepository</code> (HTTP)	quando a API existir: só a raiz de composição muda
Distância fixa nos dados	localização do aparelho (<code>geolocator</code>)	junto com a API: a distância é calculada a partir da posição do usuário
Só tema claro	<code>darkTheme</code> + <code>themeMode</code>	quando houver tempo para validar todos os contrastes no escuro (Parte 1.5)
Formatação manual (<code>replaceAll</code>)	pacote <code>intl</code>	quando o app tiver mais de um idioma
um layout para todos os tamanhos (responsivo)	layout adaptativo: <code>NavigationRail</code> na lateral em telas a partir de 600 dp	quando o app for usado em tablets ou desktop
<code>IndexedStack</code> com todas as abas	construir a aba na primeira visita	quando alguma aba ficar pesada
Saudação sem nome	<code>HomeHeader(name: ...)</code>	quando o perfil existir

Pendências do material anterior resolvidas aqui

- ☒ 6.5 Transição consistente para a Home (`fadeRoute` nos dois lugares)
- ☒ 6.6 Fundo do onboarding igual ao da Splash (via `scaffoldBackgroundColor`)
- ☒ 6.9 Código comentado da Splash removido (decisão de rota final ativa)

Pendências que continuam

- `navy 70%` nas descrições do onboarding: avalie subir para 80% (Parte 1.5).
- Espaçamentos antigos fora da grade (12, 40): migre para os tokens quando mexer nesses arquivos.
- Splash nativa com `color_dark`, mas app só com tema claro (material anterior, Parte 2.2).

Parte 9: Aplicando no seu projeto

Roteiro

1. **Promessas:** faça a tabela da Parte 0.2 com as frases do seu onboarding.
2. **Esboço:** desenhe a sua Home no papel (Parte 0.3). Para cada bloco, responda: "qual pergunta da Parte 0.1 ele responde?". Corte o que não responder a nenhuma.
3. **Contraste:** calcule a tabela da Parte 1.5 com as cores da sua marca.
4. **Tema e tokens:** `AppSpacing`, `AppRadius`, `AppTheme` e `theme:` no `MaterialApp`.
5. **Domínio:** o modelo principal do seu app, enums para conjuntos fechados, o contrato do repositório.
6. **Fake:** 4 a 6 itens variados, com atraso simulado.
7. **Shell:** `NavigationBar` com 3 a 5 destinos, `IndexedStack`, `PopScope`, abas "em breve".
8. **Navegação:** Splash e onboarding apontando para o shell, com a mesma transição.
9. **Home:** esqueleto com slivers → cabeçalho → ação principal → destaque → atalhos → lista.
10. **Estados:** carregando, erro, vazio e dados, com `Future` criado no `initState`.
11. **Qualidade:** largura máxima, fonte a 200%, leitor de tela, áreas de toque.
12. **Testes (complementar):** `flutter_test` no `pubspec`, um teste de unidade, testes dos estados.

Checklist final

- ☐ Cada promessa do onboarding tem um lugar na Home
- ☐ Nenhum elemento tocável "morto": tudo responde ao toque, nem que seja com "em breve"
- ☐ Tema centralizado no `MaterialApp`; estilos locais só para exceções justificadas
- ☐ Espaçamentos e raios vindos dos tokens

- ☐ Textos com contraste $\geq 4,5:1$; cor de destaque nunca usada como texto
- ☐ Domínio sem `import 'package:flutter/...'`
- ☐ Tela depende do **contrato** do repositório, recebido pelo construtor
- ☐ `NavigationBar` com 3 a 5 destinos, ícone + rótulo
- ☐ "Voltar" em outra aba leva ao Início
- ☐ Splash e onboarding abrem o shell com `pushReplacement` e a mesma transição
- ☐ `CustomScrollView` com slivers; lista construída sob demanda
- ☐ **Future** criado no `initState`, nunca no `build`
- ☐ Os 4 estados implementados e verificados (simulando erro e vazio)
- ☐ Puxar para atualizar funcionando, sem erro no console quando a carga falha
- ☐ Conteúdo limitado em telas largas
- ☐ Sem `overflow` com fonte a 200% em tela pequena
- ☐ Títulos de seção marcados como cabeçalho; cartões lidos de uma vez
- ☐ Áreas de toque ≥ 48 dp
- ☐ `flutter analyze` sem avisos
- ☐ (complementar) `flutter test` passando

Desafios

- Dica do dia (promessa "Informação").** Crie um cartão com uma dica curta sobre leitura de rótulos ("Maltodextrina pode conter glúten? Veja como identificar."). Os dados vêm de um `TipRepository` com `fake`. Onde ele entra na hierarquia da Parte 0.3, e por quê?
- Esconder o cartão de perfil.** Crie um `ProfileStorage` (como o `OnboardingStorage`) com `isCompleted()`. A Home mostra o `ProfilePromptCard` só para quem não completou. Escreva um teste de widget para os dois casos.
- Filtro por categoria.** Ao tocar em uma categoria, em vez do "em breve", filtre a lista "Perto de você". Dica: guarde a `EstablishmentCategory?` selecionada no estado e filtre os dados no `builder`, e não no repositório. Como o usuário desfaz o filtro?
- Esqueleto animado.** Faça os blocos de carregamento "pulsarem" suavemente com um `AnimationController` em repetição (`repeat(reverse: true)`) e um `FadeTransition`. Lembre-se do `dispose`.
- Tema escuro.** Crie `AppTheme.dark` e refaça a tabela de contraste. O navy funciona como fundo? O dourado funciona como texto sobre ele?

Apêndice: Solução de problemas

Erros que o `flutter analyze` costuma mostrar ao seguir este material, e onde está a correção:

Mensagem	Causa provável	Correção
Undefined name '_nearbyFuture'	faltou o campo e o <code>initState</code>	Parte 5.6
The declaration '_buildNearby' isn't referenced (aviso)	o <code>build</code> ainda não chama <code>_buildNearby()</code>	Parte 5.9
Missing concrete implementation of 'State.build', seguido de vários Undefined name '_...'	uma <code>}</code> a mais fechou a classe antes da hora (geralmente em <code>_finishOnboarding</code>)	Parte 3.5
Undefined name 'completed'	faltou <code>final completed = await OnboardingStorage().isCompleted();</code> na Splash	Parte 3.5
Undefined name 'OnboardingStorage'	o <code>import</code> de <code>onboarding_storage.dart</code> foi removido da Splash	Parte 3.5
Target of URI doesn't exist:	arquivo criado em outra pasta	Parte 0.5: <code>lib/app/routes/</code> e <code>lib/features/home/presentation/widgets/</code>

Mensagem	Causa provável	Correção
'.../fade_route.dart' ou '.../nearby_states.dart'		
Methods can't be invoked in constant expressions / The constructor being called isn't a const constructor / Invalid constant value	children: const [com itens que não são constantes	Parte 4.3
cannot run without a dependency on flutter_test	falta flutter_test no pubspec.yaml	Parte 7.2

Dica: quando aparecerem **muitos** erros de uma vez em um arquivo que antes compilava, procure primeiro uma chave ou um parêntese desbalanceado perto da **primeira** linha com erro. Os outros erros costumam ser consequência dele.

Apêndice: Glossário

Termo	Significado
Shell	Moldura fixa do app (aqui, a barra de navegação) que troca o conteúdo do meio
ThemeData	Conjunto de estilos padrão do app, aplicado pelo MaterialApp
ColorScheme	Cores do tema organizadas por papel (primary, surface, onPrimary...)
ColorScheme.fromSeed	Gera uma paleta completa a partir de uma cor-semente
Subtema	Estilo padrão de um componente (CardThemeData, NavigationBarThemeData...)
Contraste (WCAG)	Razão de luminosidade entre texto e fundo; mínimo 4,5:1 para texto comum
Enum com campos	Enum cujos valores carregam dados (ex.: label)
Repositório	Objeto que obtém os dados de um assunto, escondendo a origem deles
abstract interface class	Contrato: não pode ser instanciado e deve ser implementado com implements
Fake	Implementação simplificada e funcional de um contrato, usada no desenvolvimento
Stub	Substituto usado em testes, que devolve respostas pré-definidas
Raiz de composição	Ponto onde as implementações concretas são escolhidas e conectadas
NavigationBar	Barra de navegação inferior do Material 3
IndexedStack	Mostra um filho por vez, mantendo todos vivos (preserva o estado das abas)
PopScope	Controla o que acontece com o gesto/botão "voltar"
Sliver	Pedaço de uma área rolável com comportamento próprio de rolagem
CustomScrollView	Área rolável composta por slivers
SliverAppBar	Barra superior que participa da rolagem (pinned, floating)
SliverList.list / .separated	Listas em forma de sliver: fixa / sob demanda com separadores
SliverToBoxAdapter	Coloca um widget comum dentro de um CustomScrollView
Extensão	Acrescenta membros a um tipo existente sem alterá-lo
switch exaustivo	switch em que o compilador exige que todos os casos sejam tratados
ValueChanged<T>	Função que recebe um valor do tipo T e não devolve nada

Termo	Significado
Collection if / for	<code>if</code> e <code>for</code> dentro de listas de widgets
Spread (...)	"Despeja" os itens de uma lista dentro de outra
FutureBuilder	Reconstrói a interface conforme o andamento de um <code>Future</code>
AsyncSnapshot	Estado de um <code>Future</code> no <code>FutureBuilder</code> (<code>connectionState</code> , <code>data</code> , <code>error</code>)
Esqueleto (skeleton)	Formas que antecipam o conteúdo durante o carregamento
RefreshIndicator	"Puxar para atualizar"
Microcopy	Textos curtos da interface: rótulos de botões, dicas de campos, mensagens de erro e de lista vazia
SnackBar / ScaffoldMessenger	Mensagem temporária na base da tela / quem a exibe
Operador cascata (..)	Várias chamadas no mesmo objeto
MergeSemantics	Junta os textos de um grupo em um único item para o leitor de tela
Espaço não separável (\u00A0)	Espaço que impede quebra de linha ("1,2 km")
Teste de unidade / de widget	Verifica uma função isolada / uma tela em ambiente simulado
pumpAndSettle	Avança o tempo no teste até não haver mais nada acontecendo